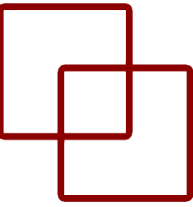


Diseño de redes neuronales



Dr. Manuel Castillo-Cara
Deep Learning con Python y Keras
Web: www.smartcityperu.org

Índice

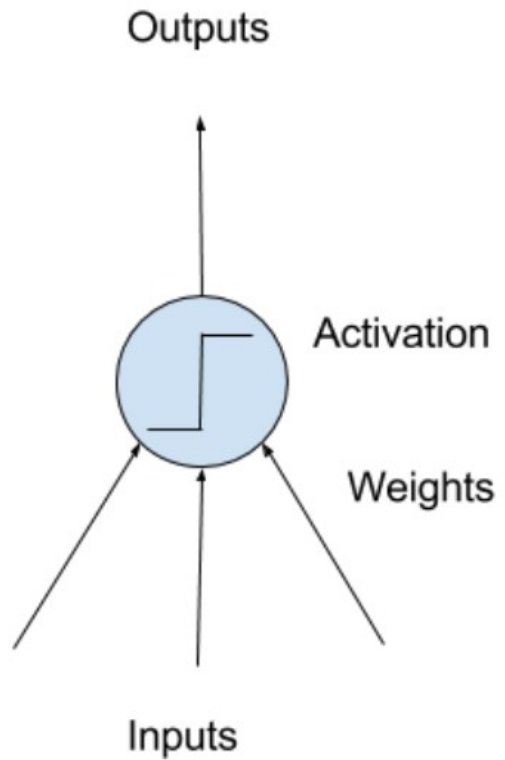
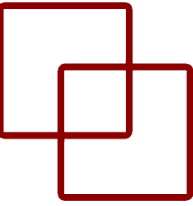


- Background
- Red Neuronal Artificial
- Gradiente Descendiente
- Perceptron Multicapa / Redes Neuronales
- Redes *Feed Forward* y *Recurrent Neural Networks*
- Pesos, parámetros sesgo y activación
- Funciones de activación: Sigmoidal y ReLu
- Funciones de activación: Softmax
- Gradiente descendiente y Backpropagation

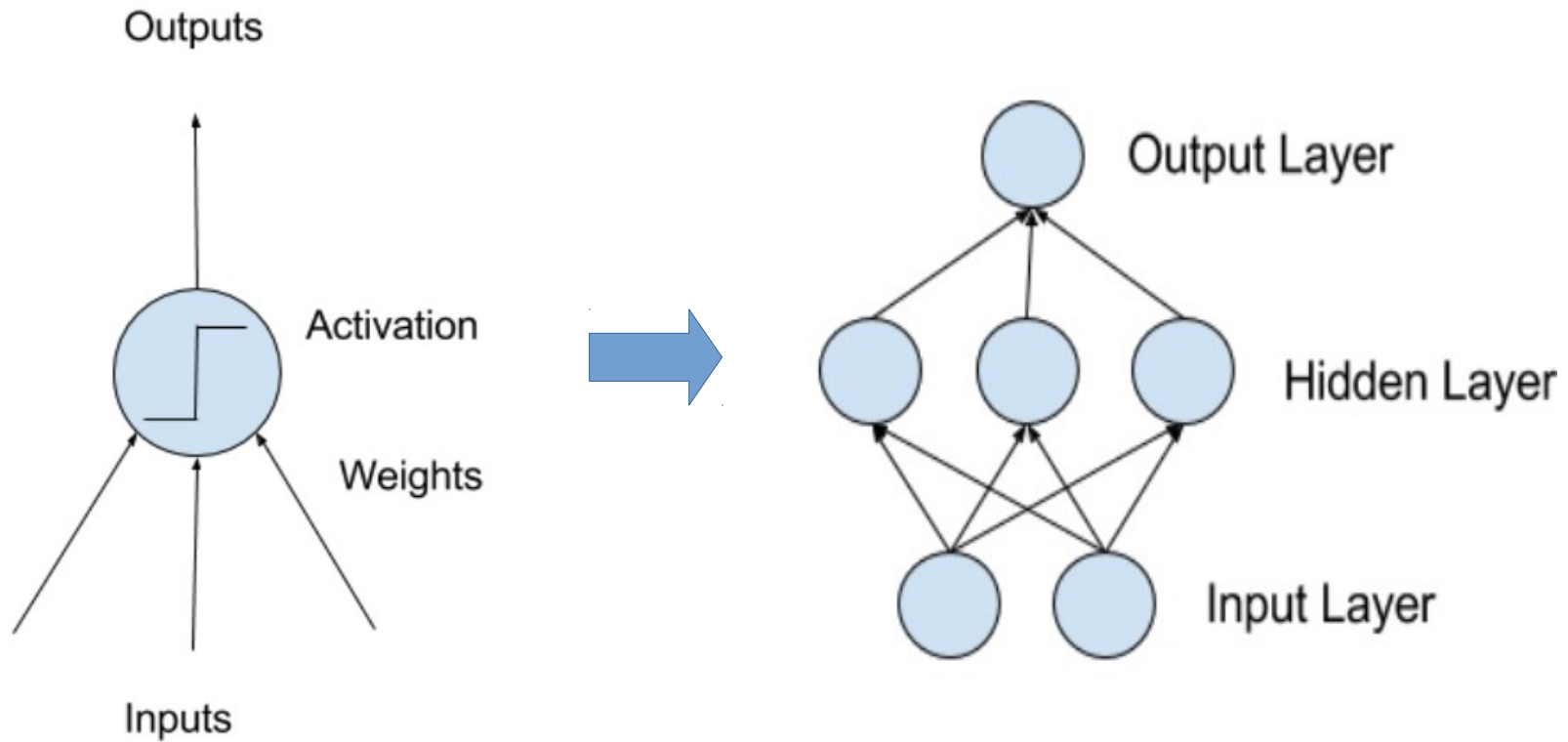
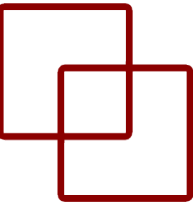


Background

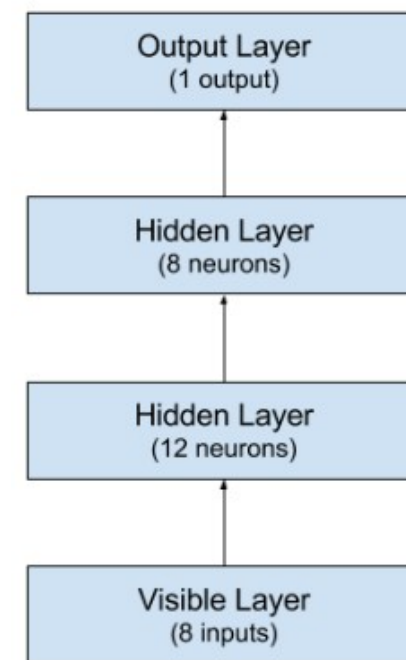
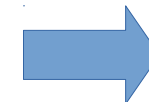
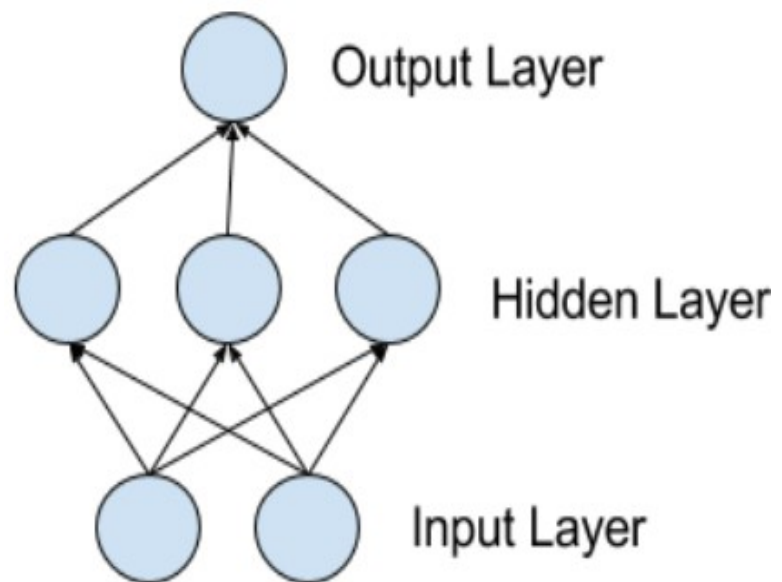
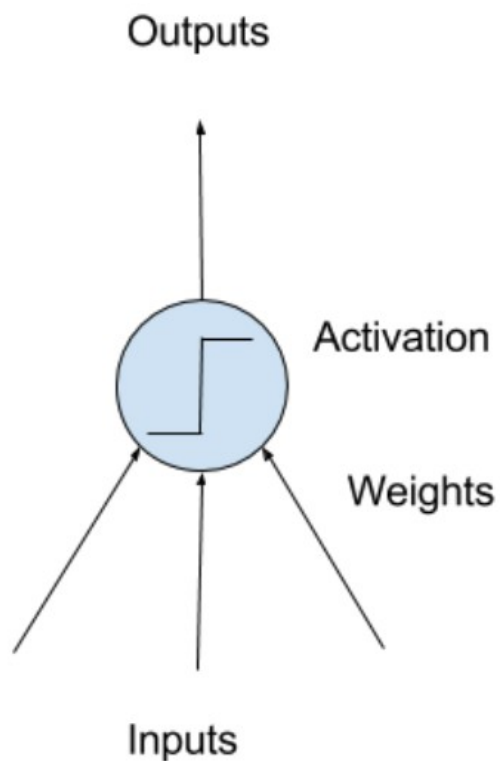
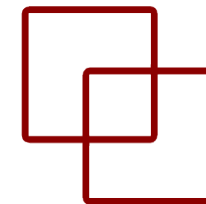
0. Background



0. Background



0. Background

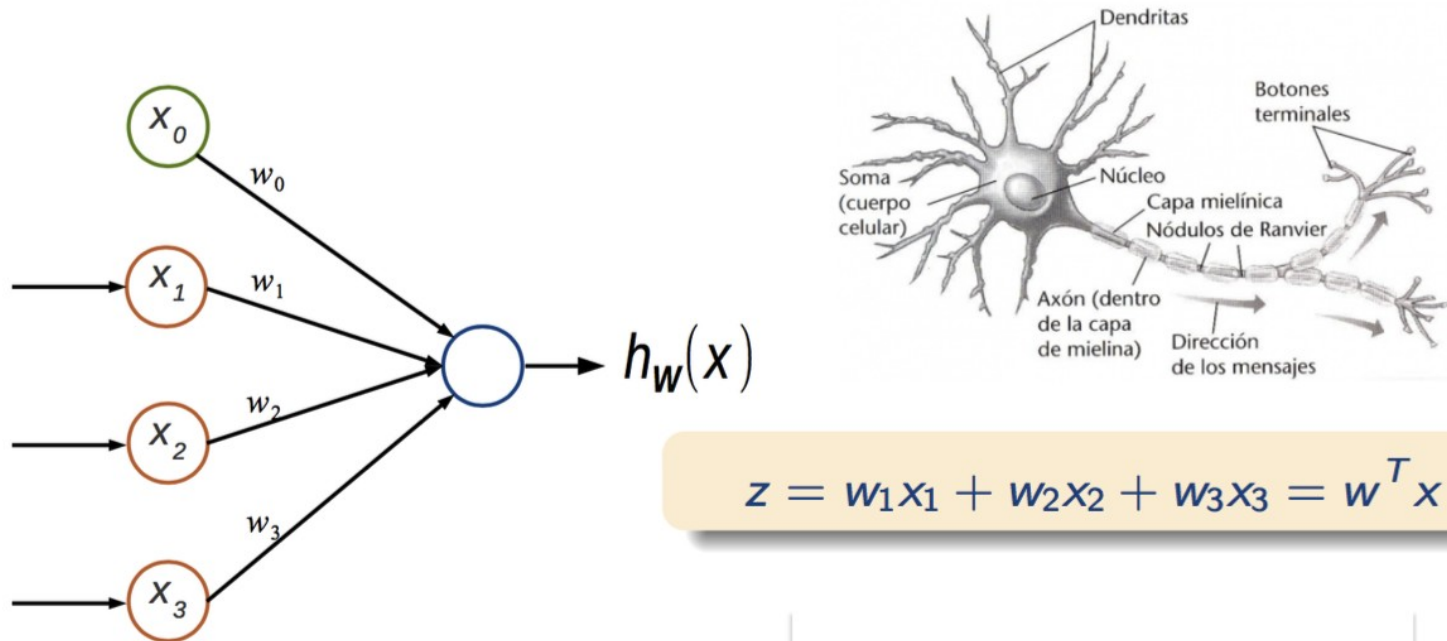




Red Neuronal Artificial

1. Red neuronal artificial

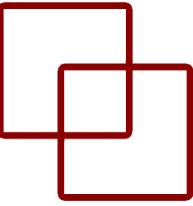
- Las RNs artificiales surgen inicialmente como un modo de simular el comportamiento de las redes neuronales naturales.



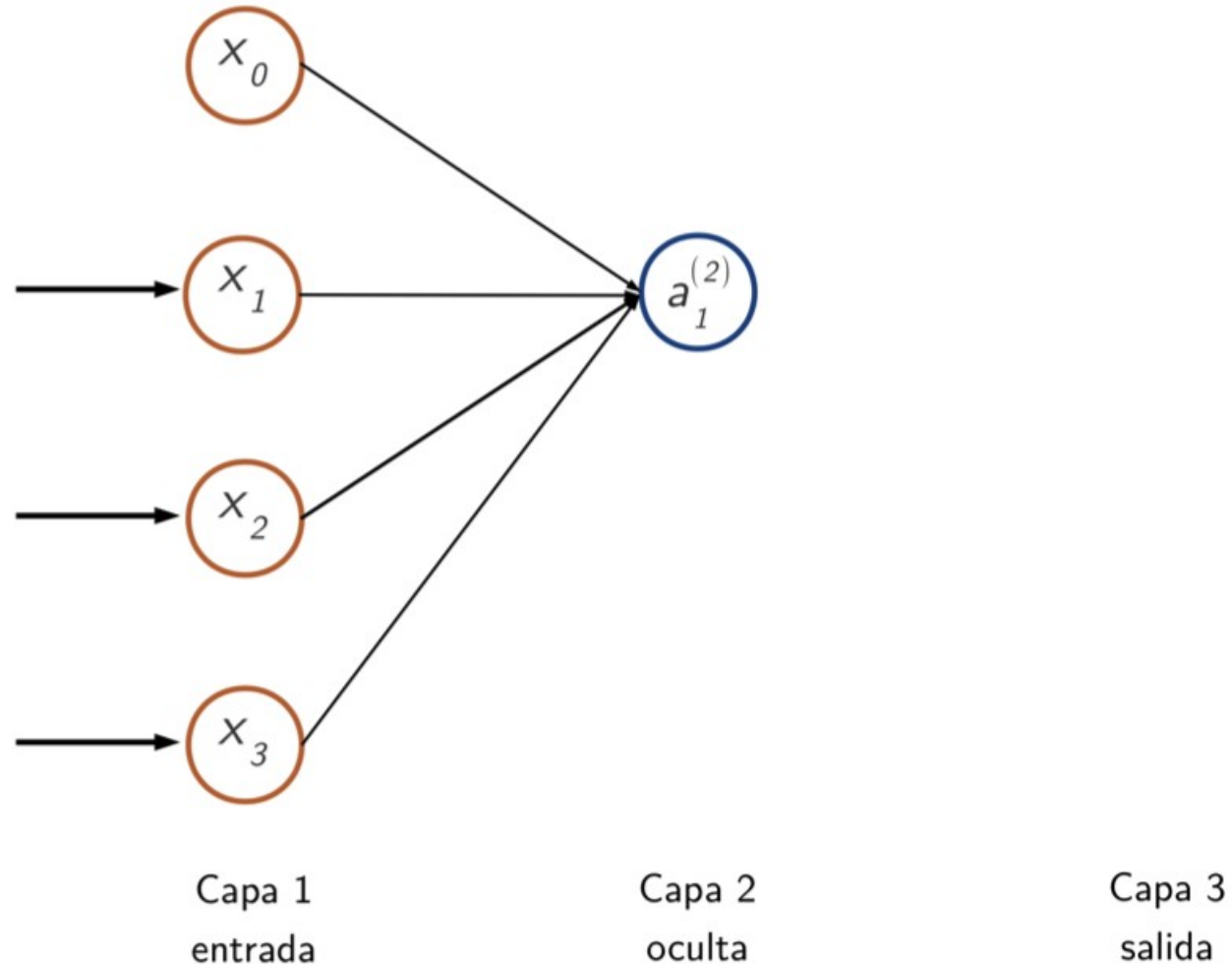
$$z = w_1x_1 + w_2x_2 + w_3x_3 = w^T x$$

$$h_w(x) = \frac{1}{1 + e^{-w^T x}}$$

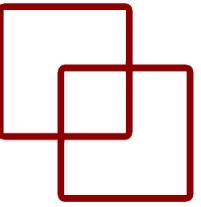
2. Red neuronal



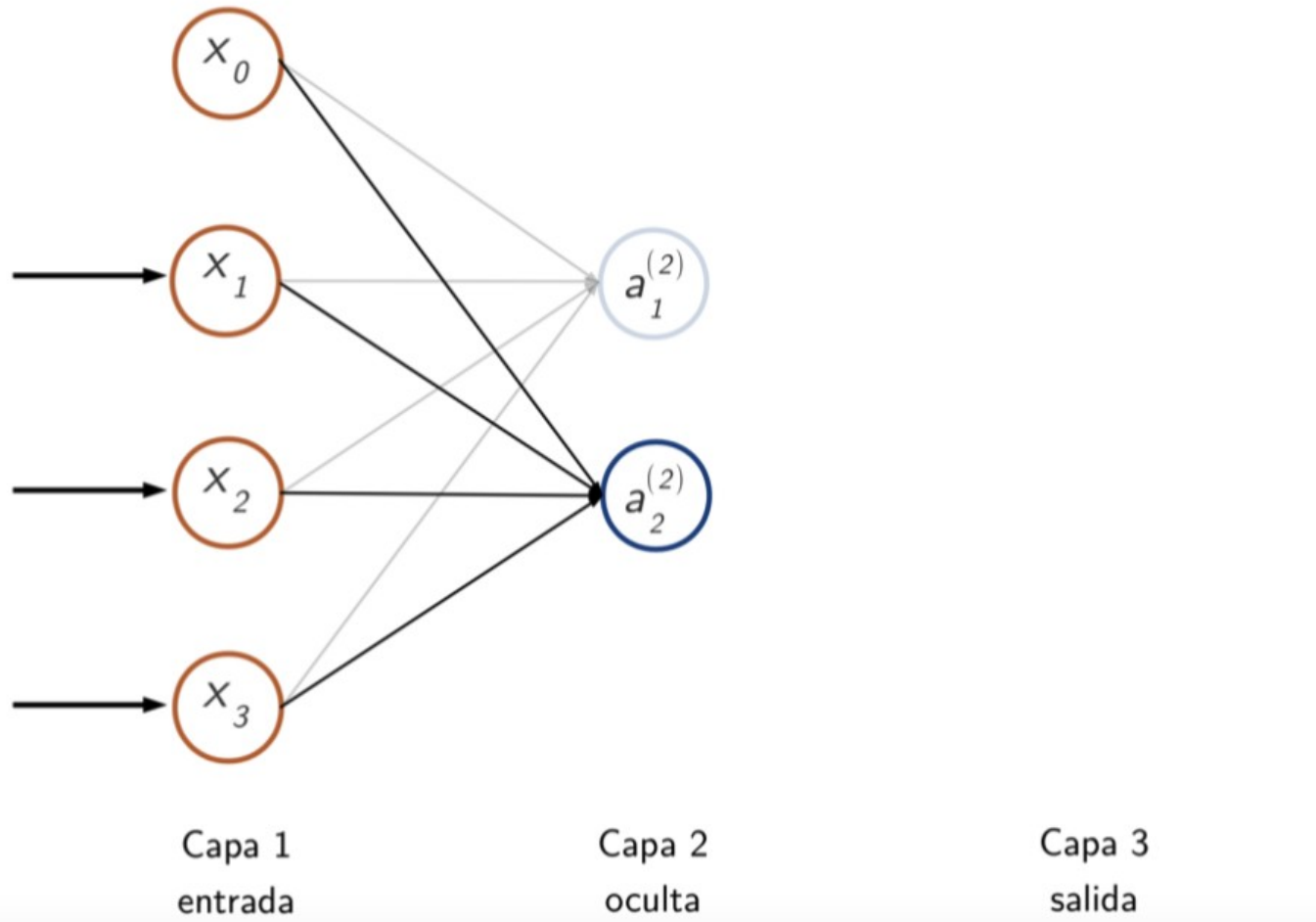
- Red formada por varias capas de neuronas artificiales interconectadas.



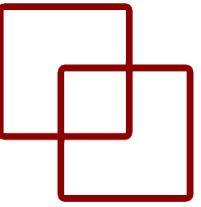
2. Red neuronal



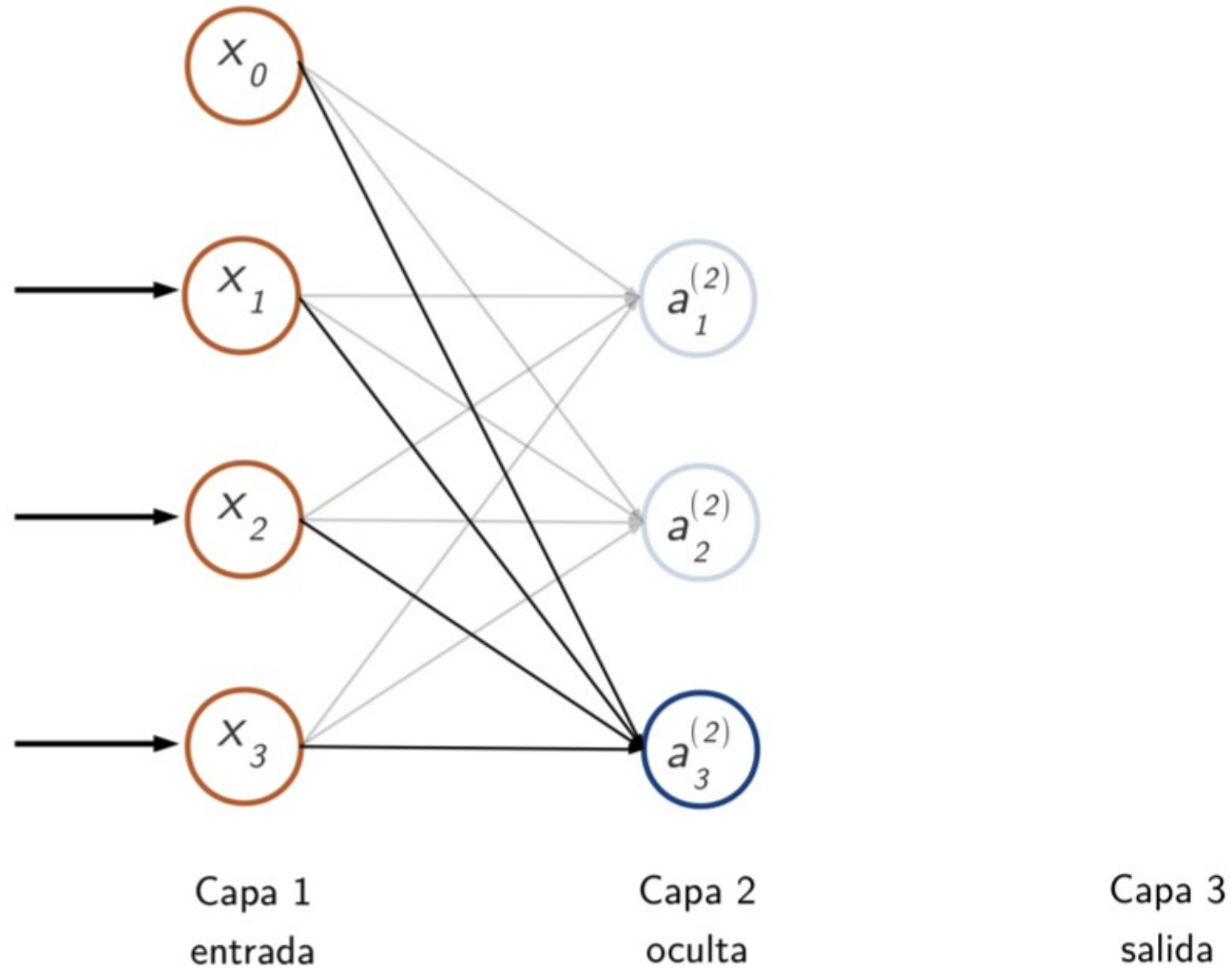
- Red formada por varias capas de neuronas artificiales interconectadas.



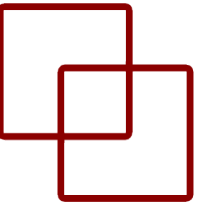
2. Red neuronal



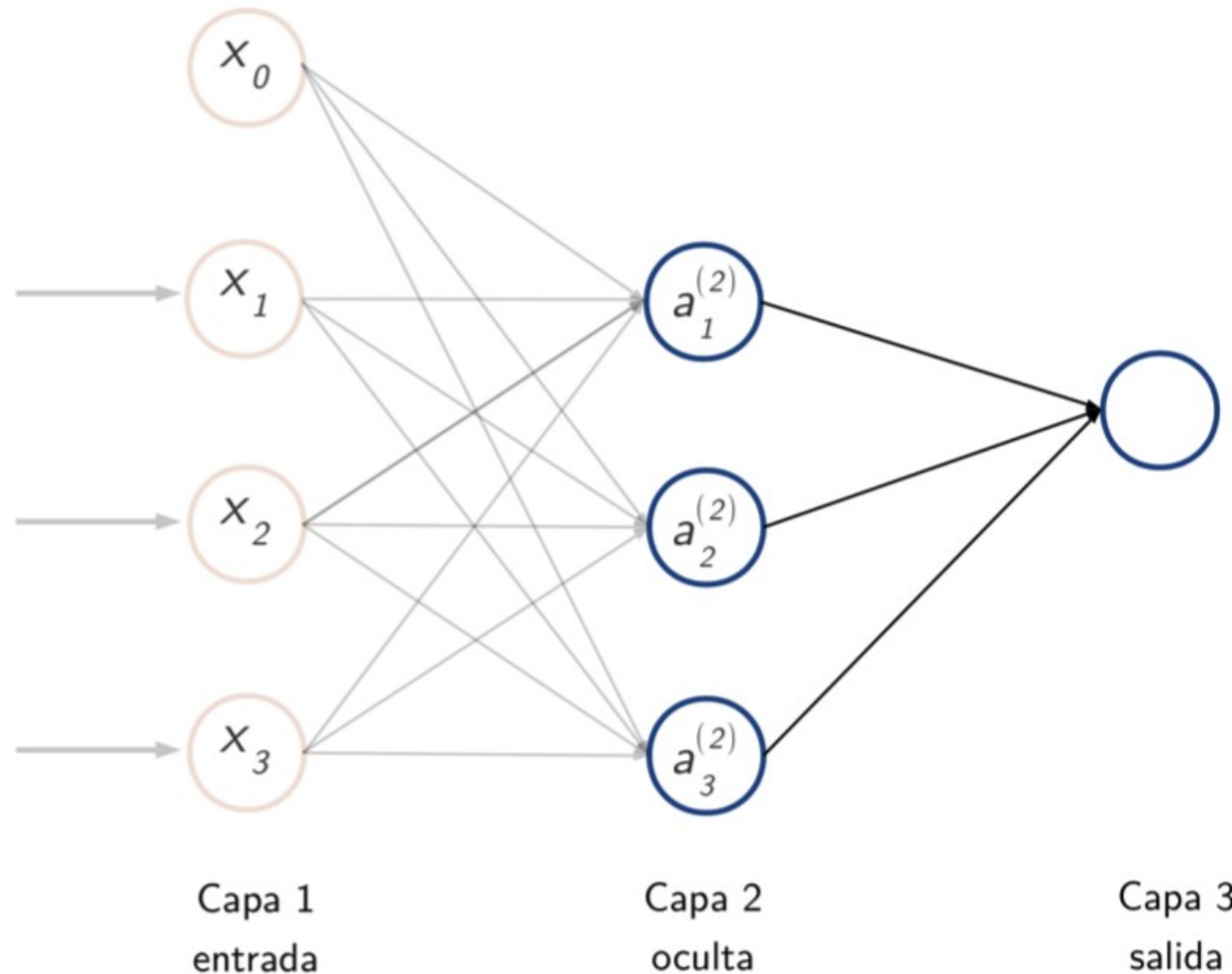
- Red formada por varias capas de neuronas artificiales interconectadas.



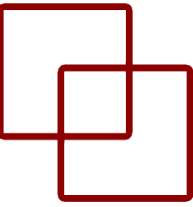
2. Red neuronal



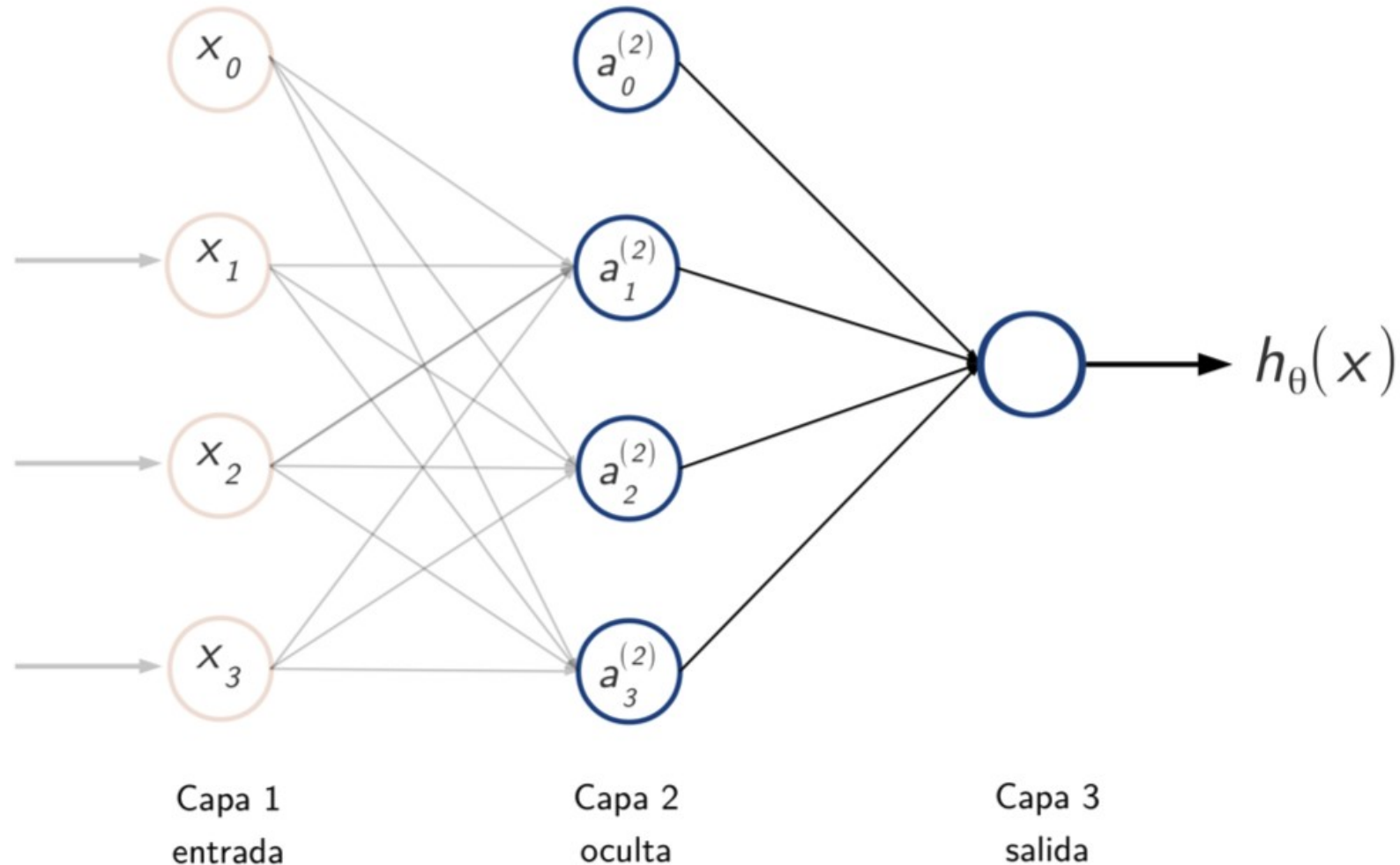
- Red formada por varias capas de neuronas artificiales interconectadas.



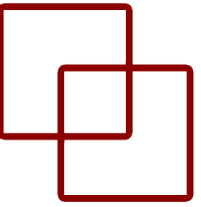
2. Red neuronal



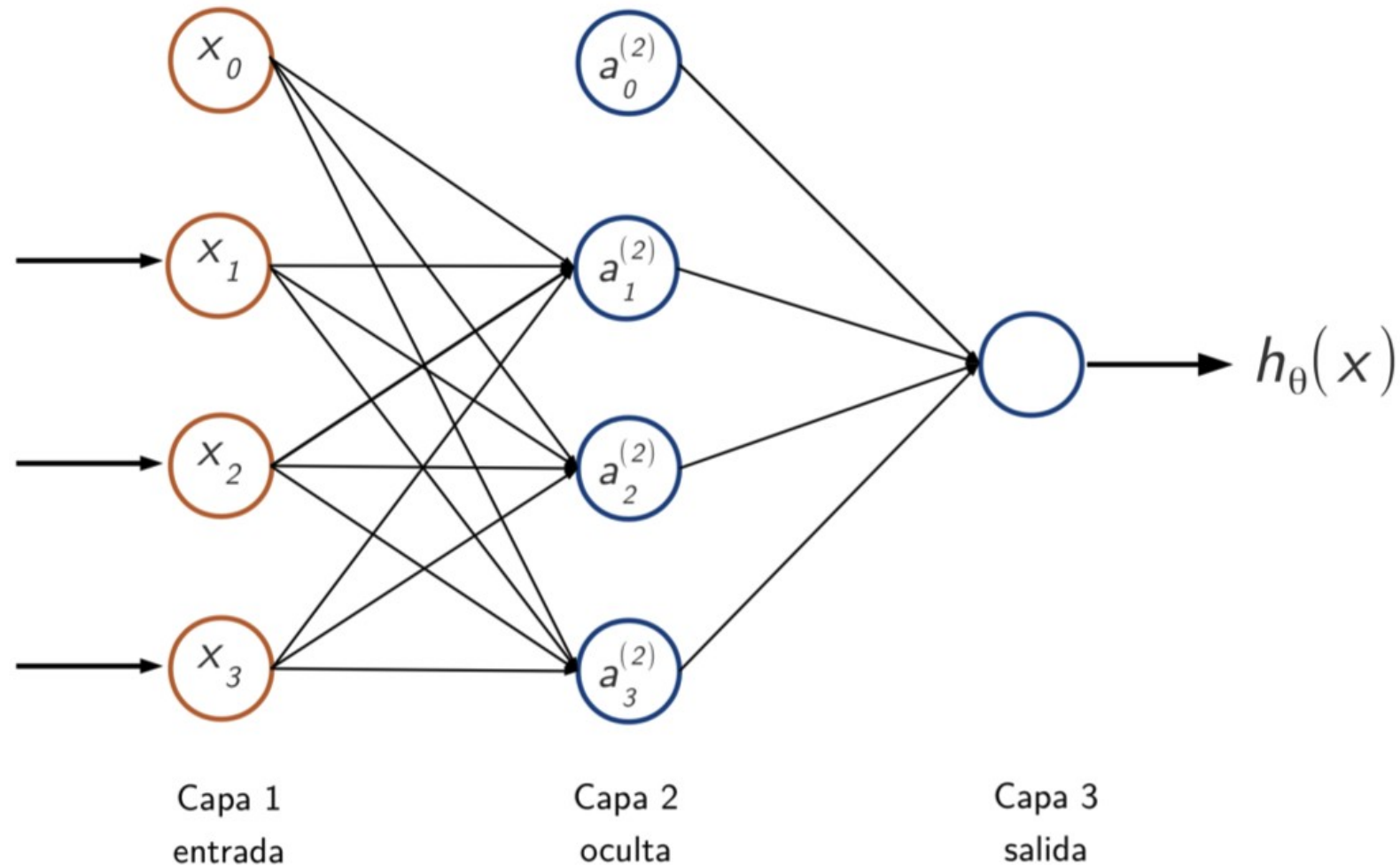
- Red formada por varias capas de neuronas artificiales interconectadas.



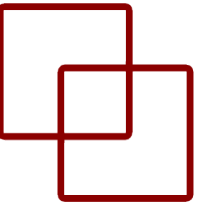
2. Red neuronal



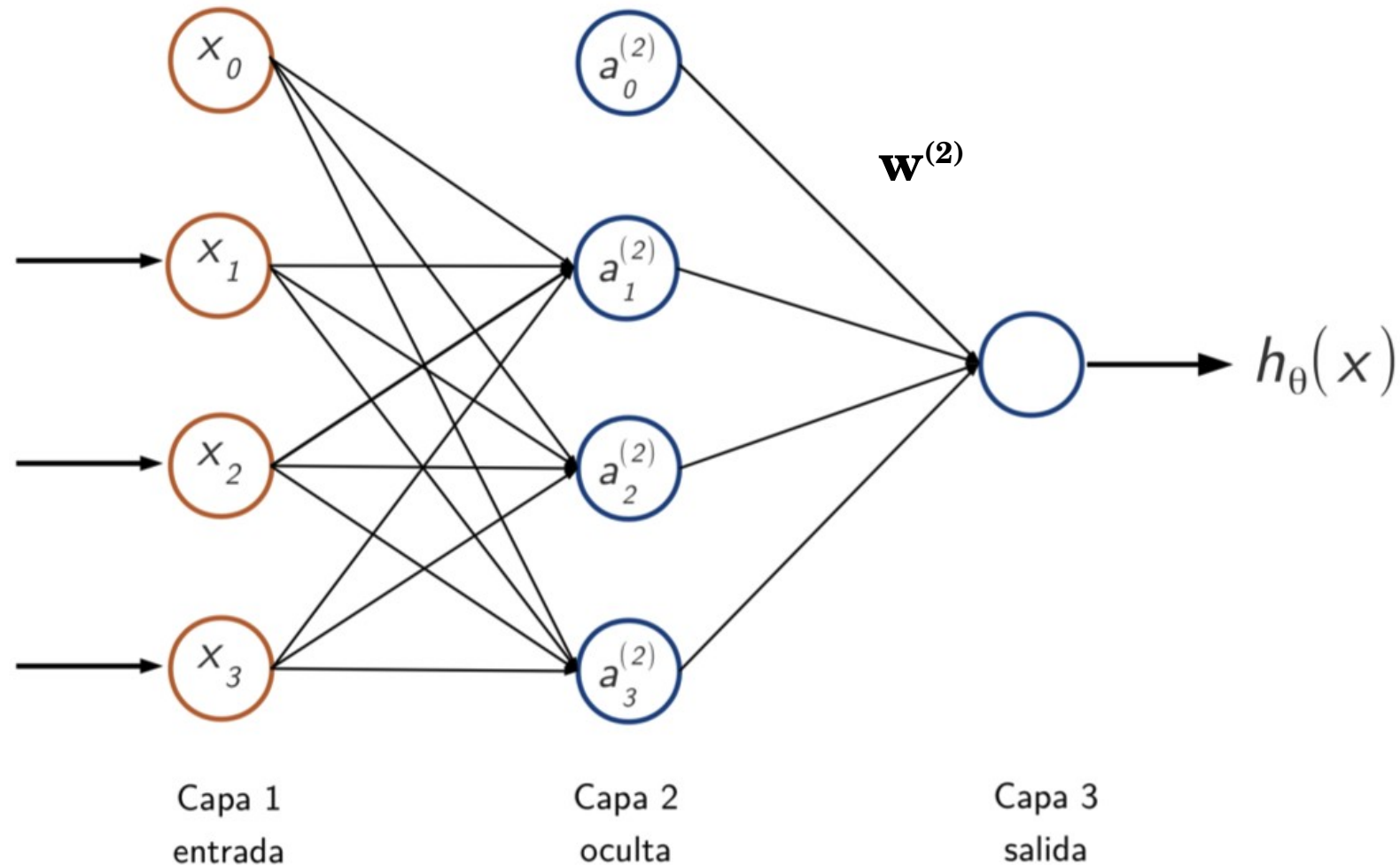
- Red formada por varias capas de neuronas artificiales interconectadas.



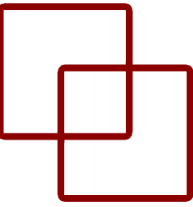
2. Red neuronal



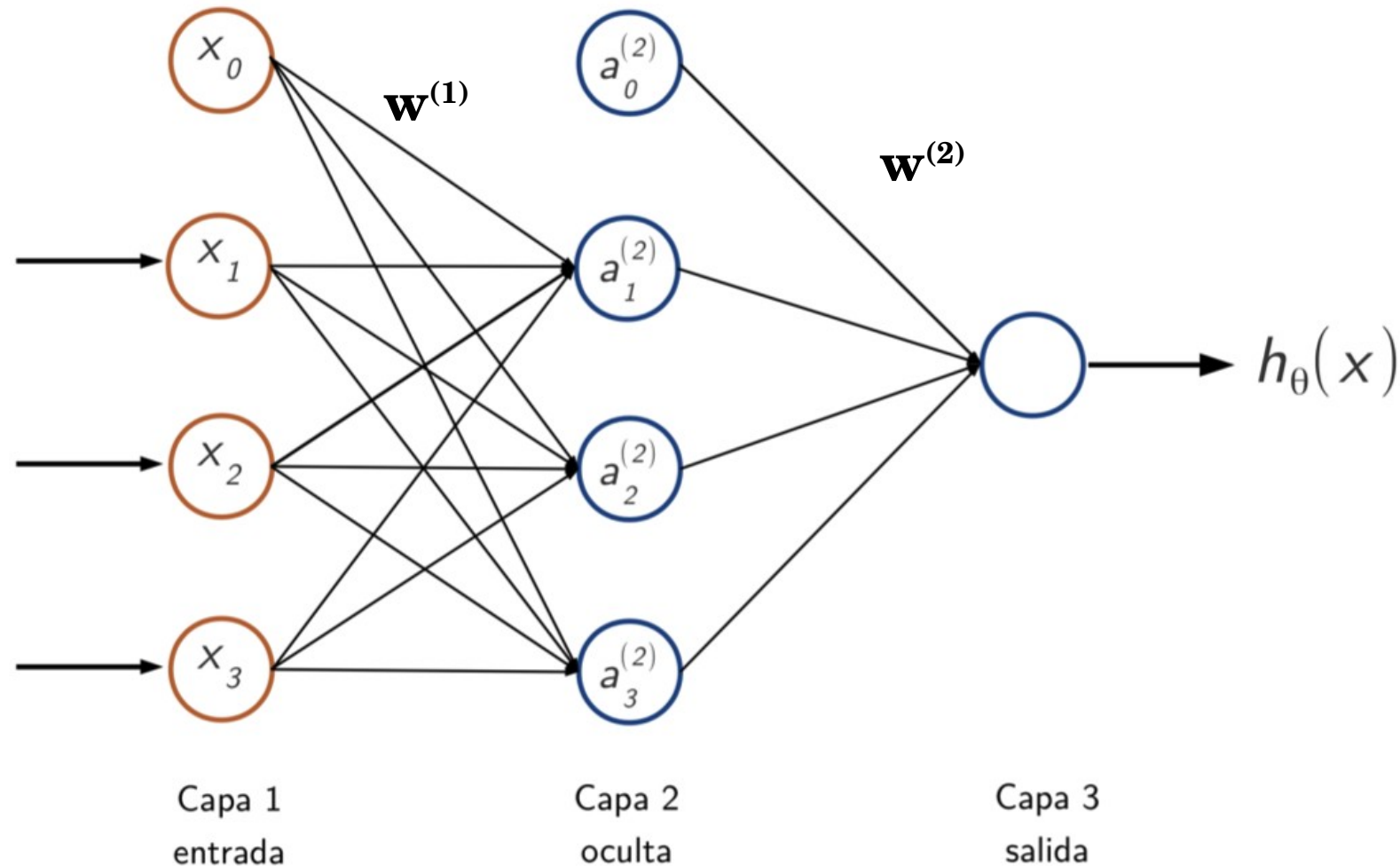
- Red formada por varias capas de neuronas artificiales interconectadas.



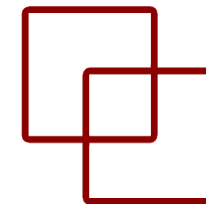
2. Red neuronal



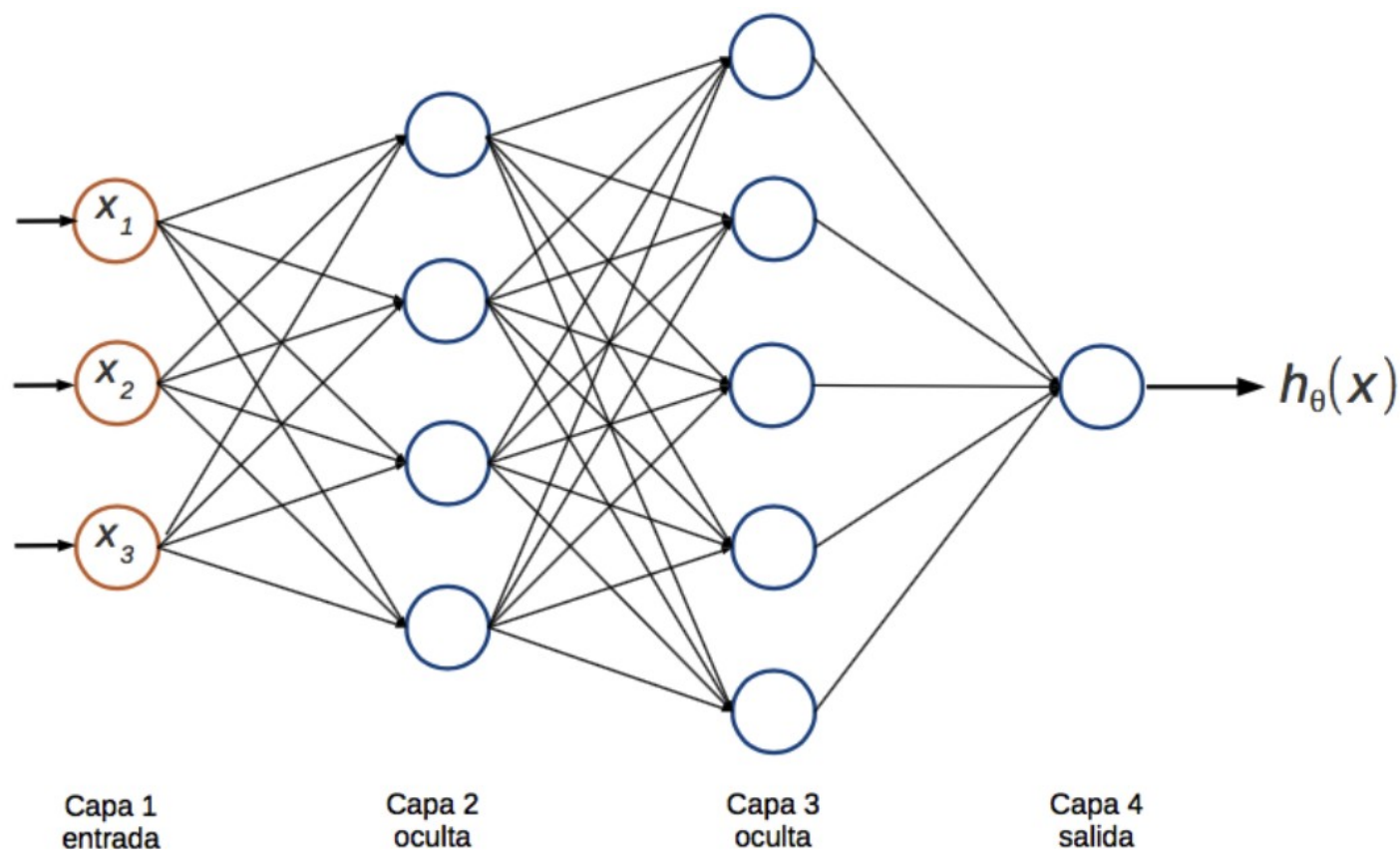
- Red formada por varias capas de neuronas artificiales interconectadas.



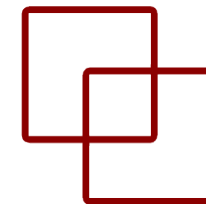
3. Multilayer Perceptrón



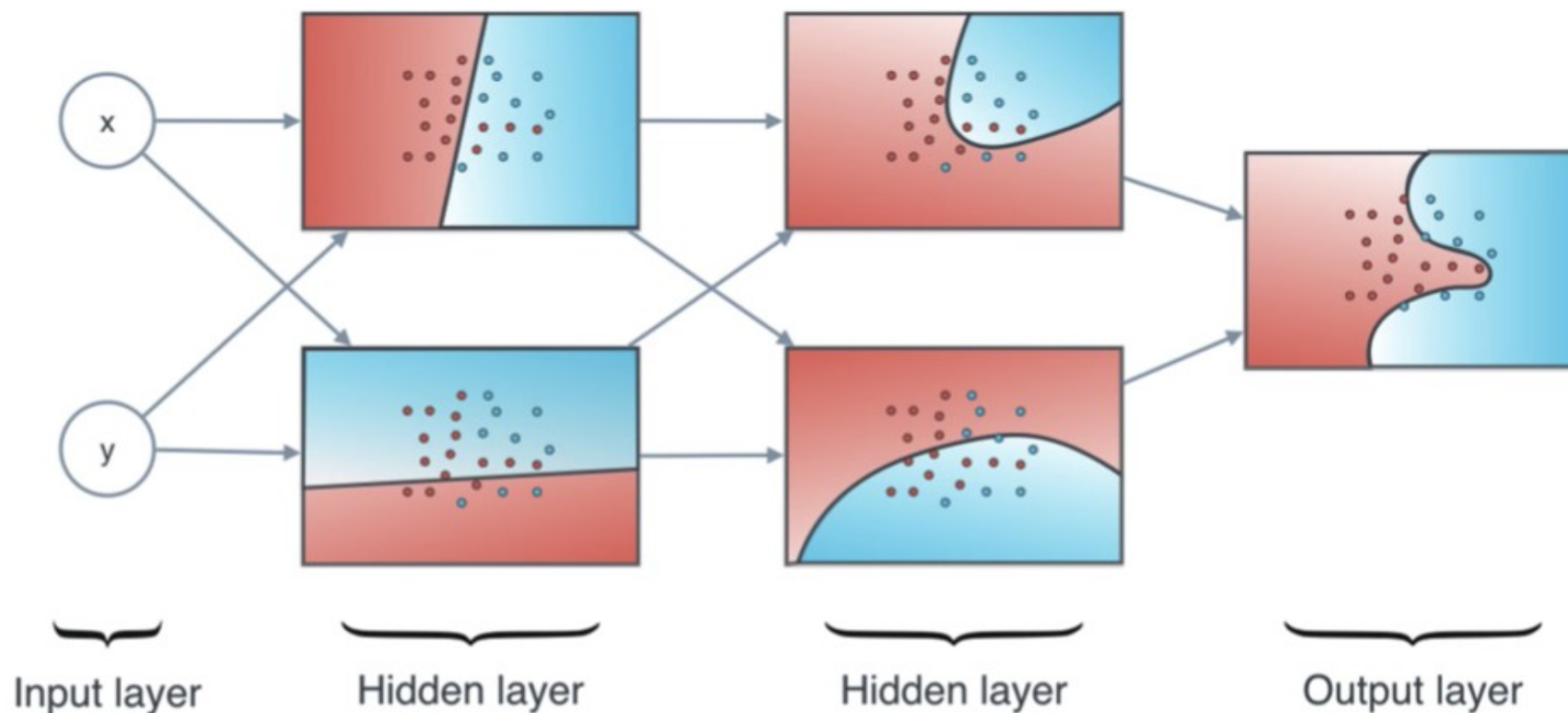
- Hay muchos tipos de redes basados en neuronas artificiales, el más clásico se denomina **perceptrón multicapa**.



4. Aplicaciones

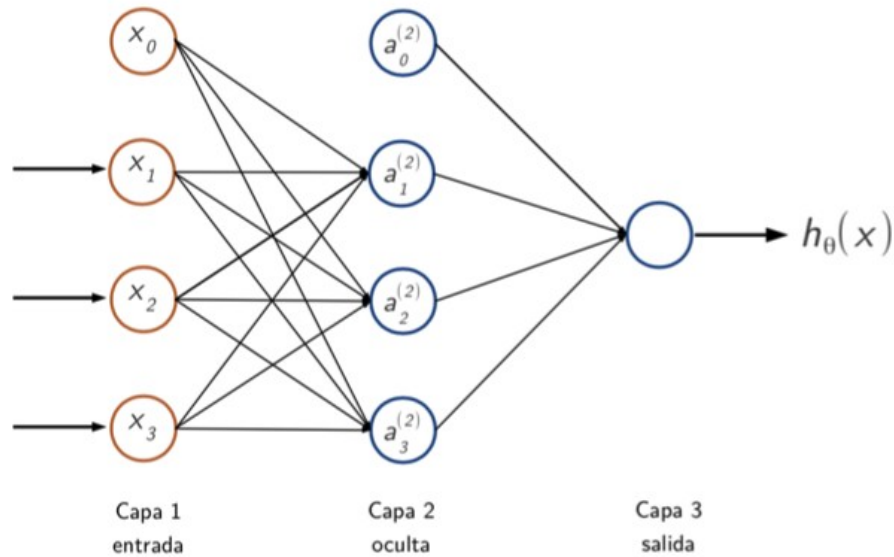
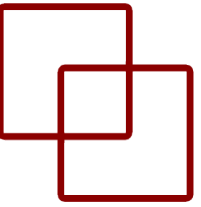


- Son la base de las **redes profundas**, actualmente muy utilizadas con mucho éxito.
- Cada **capa extrae características** cada vez más complejas



5. Propagación hacia delante

Forward



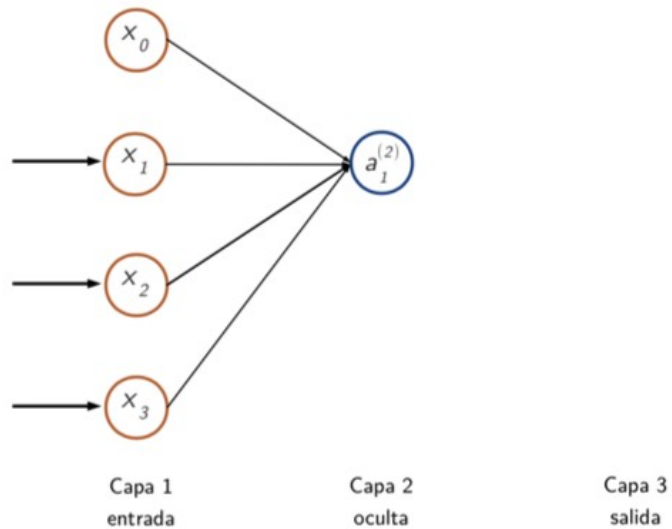
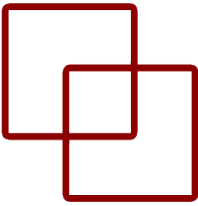
$$\mathbf{w}^{(1)} = \begin{bmatrix} 0.6 & -0.1 & -1.0 & -3.0 \\ -2.2 & -3.4 & 0.2 & -2.6 \\ 4.3 & 3.2 & -4.6 & 2.8 \end{bmatrix}$$

$$\mathbf{w}^{(2)} = \begin{bmatrix} 3.8 & -0.1 & -5.0 & 2.2 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ -2 \\ 1 \\ 2 \end{bmatrix} \Rightarrow$$

5. Propagación hacia delante

Forward



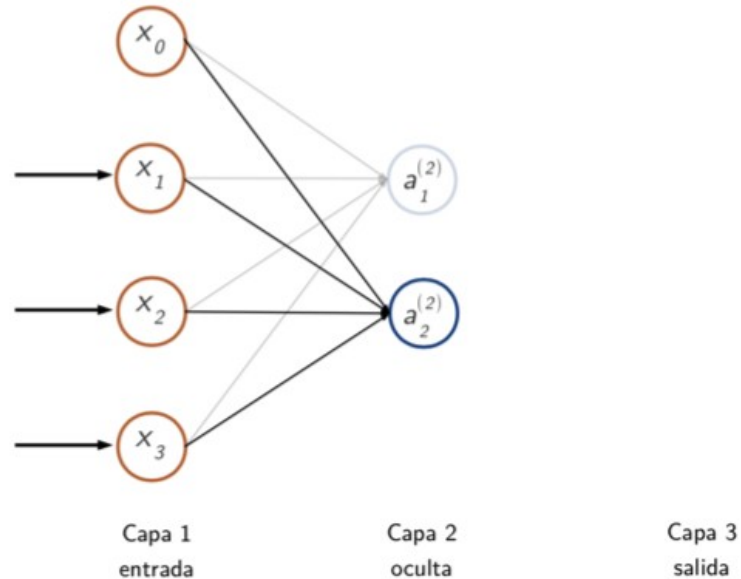
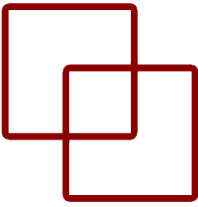
$$\mathbf{w}^{(1)} = \begin{bmatrix} 0.6 & -0.1 & -1.0 & -3.0 \\ -2.2 & -3.4 & 0.2 & -2.6 \\ 4.3 & 3.2 & -4.6 & 2.8 \end{bmatrix}$$

$$\mathbf{w}^{(2)} = \begin{bmatrix} 3.8 & -0.1 & -5.0 & 2.2 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ -2 \\ 1 \\ 2 \end{bmatrix} \Rightarrow \begin{aligned} z_1^{(2)} &= (0.6 \cdot 1) + (-0.1 \cdot -2) + (-1.0 \cdot 1) + (-3.0 \cdot 2) = -6.2 \\ a_1^{(2)} &= \frac{1}{1 + e^{-z_1^{(2)}}} = \frac{1}{1 + e^{-(-6.2)}} = 0.0020 \end{aligned} \quad \left| \quad a_1^{(2)} = 0.00$$

5. Propagación hacia delante

Forward



$$\mathbf{w}^{(1)} = \begin{bmatrix} 0.6 & -0.1 & -1.0 & -3.0 \\ -2.2 & -3.4 & 0.2 & -2.6 \\ 4.3 & 3.2 & -4.6 & 2.8 \end{bmatrix}$$

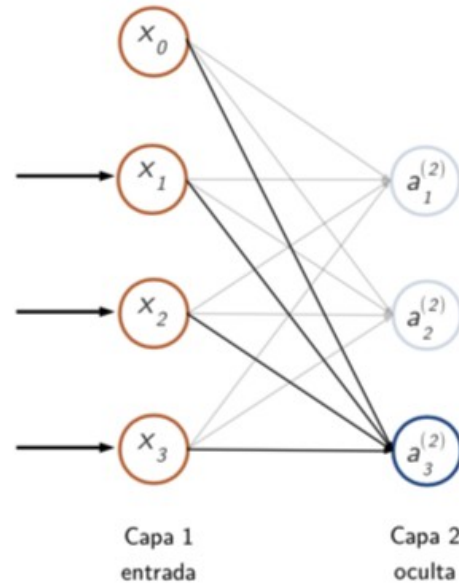
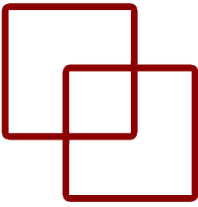
$$\mathbf{w}^{(2)} = \begin{bmatrix} 3.8 & -0.1 & -5.0 & 2.2 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ -2 \\ 1 \\ 2 \end{bmatrix} \Rightarrow \begin{aligned} z_2^{(2)} &= (-2.2 \cdot 1) + (-3.4 \cdot -2) + (0.2 \cdot 1) + (-2.6 \cdot 2) = -0.4 \\ a_2^{(2)} &= \frac{1}{1 + e^{-z_2^{(2)}}} = \frac{1}{1 + e^{-(-0.4)}} = 0.40 \end{aligned}$$

$$\begin{aligned} a_1^{(2)} &= 0.00 \\ a_2^{(2)} &= 0.40 \end{aligned}$$

5. Propagación hacia delante

Forward



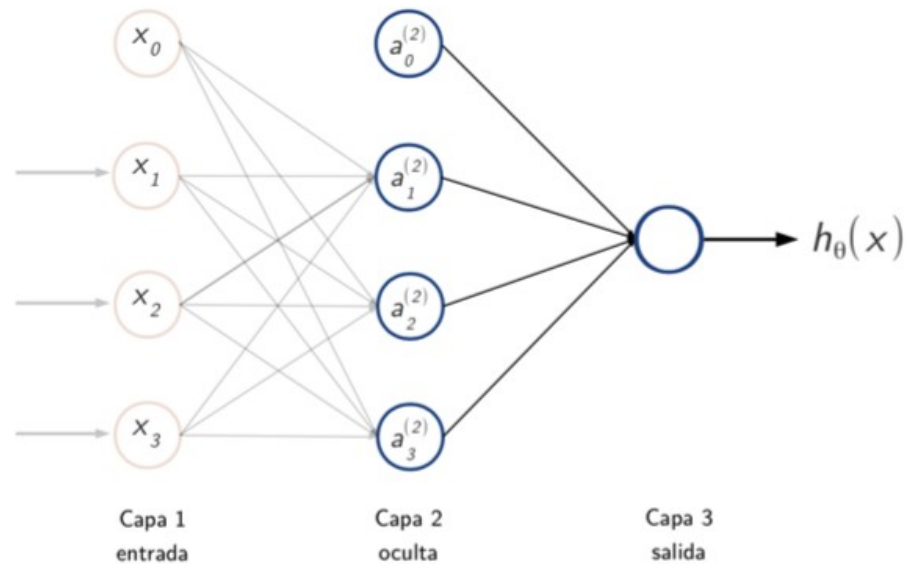
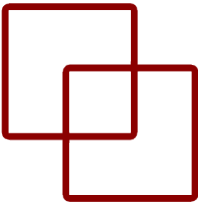
$$\mathbf{w}^{(1)} = \begin{bmatrix} 0.6 & -0.1 & -1.0 & -3.0 \\ -2.2 & -3.4 & 0.2 & -2.6 \\ \mathbf{4.3} & \mathbf{3.2} & \mathbf{-4.6} & \mathbf{2.8} \end{bmatrix}$$

$$\mathbf{w}^{(2)} = \begin{bmatrix} 3.8 & -0.1 & -5.0 & 2.2 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ -2 \\ 1 \\ 2 \end{bmatrix} \Rightarrow \begin{aligned} z_3^{(2)} &= (4.3 \cdot 1) + (3.2 \cdot -2) + (-4.6 \cdot 1) + (2.8 \cdot 2) = -1.1 \\ a_3^{(2)} &= \frac{1}{1 + e^{-z_3^{(2)}}} = \frac{1}{1 + e^{-(-1.1)}} = 0.25 \end{aligned} \left| \begin{aligned} a_1^{(2)} &= 0.00 \\ a_2^{(2)} &= 0.40 \\ a_3^{(2)} &= 0.25 \end{aligned} \right.$$

5. Propagación hacia delante

Forward



$$\mathbf{w}^{(1)} = \begin{bmatrix} 0.6 & -0.1 & -1.0 & -3.0 \\ -2.2 & -3.4 & 0.2 & -2.6 \\ 4.3 & 3.2 & -4.6 & 2.8 \end{bmatrix}$$

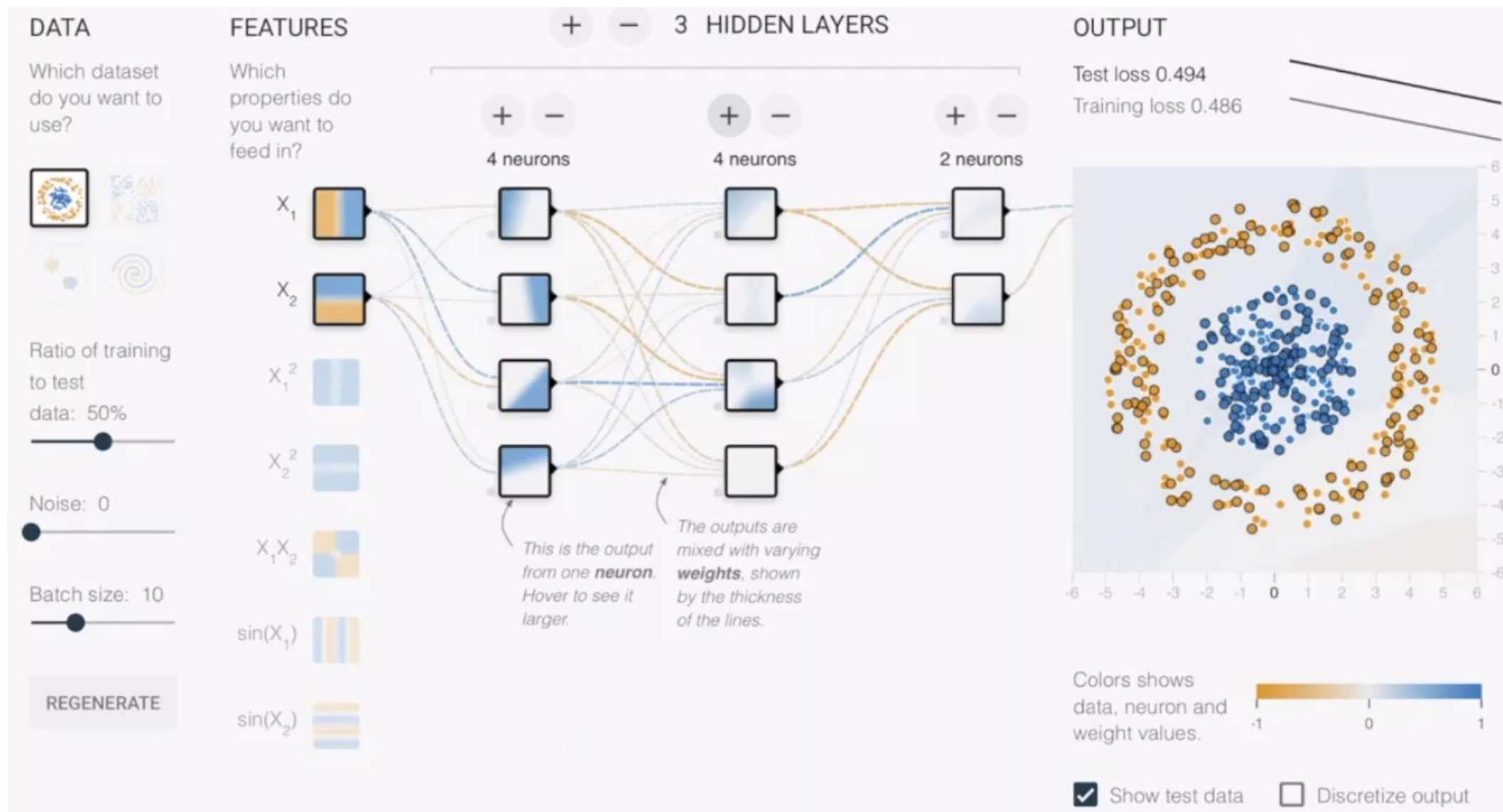
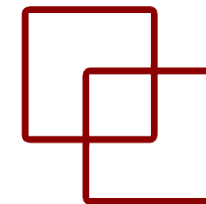
$$\mathbf{w}^{(2)} = [\mathbf{3.8} \quad \mathbf{-0.1} \quad \mathbf{-5.0} \quad \mathbf{2.2}]$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ -2 \\ 1 \\ 2 \end{bmatrix} \Rightarrow a_1^{(3)} = \frac{1}{1+e^{-z_1^{(3)}}} = \frac{1}{1+e^{-2.35}} = 0.91$$

$h_{\mathbf{w}}(\mathbf{x}) = 0.91$

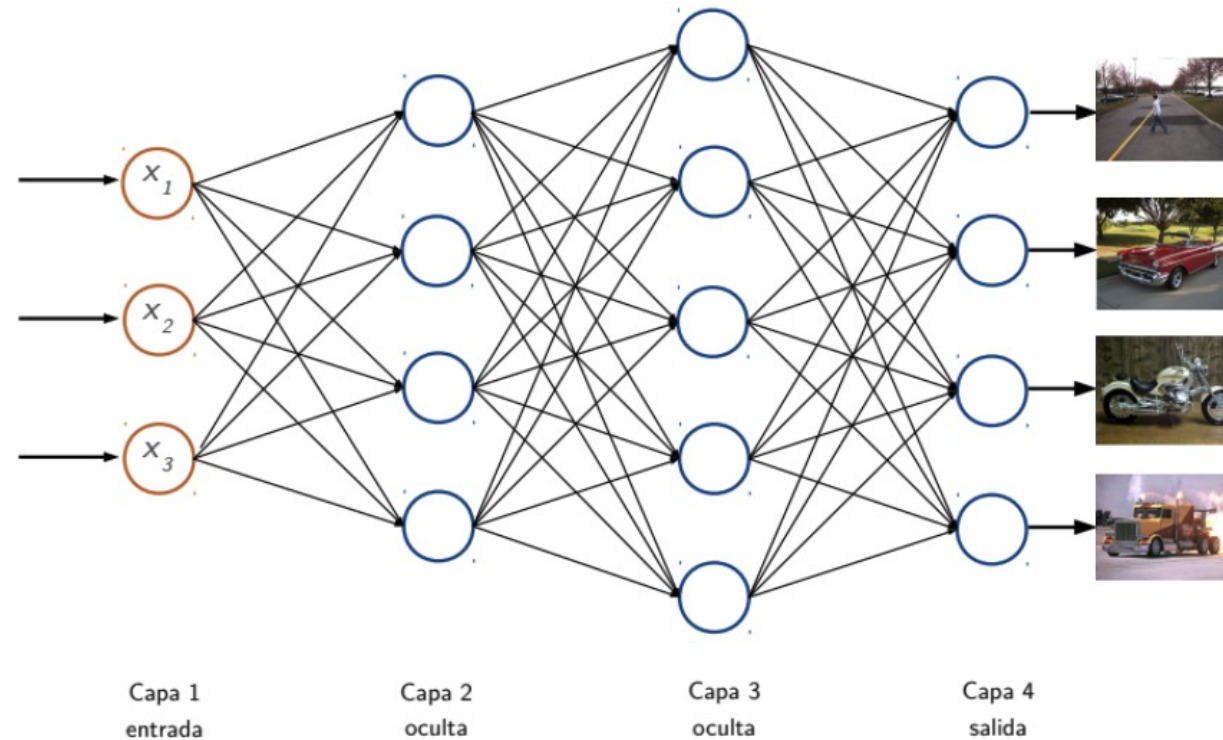
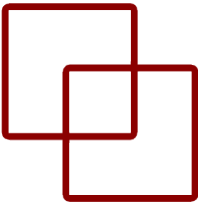
$$\begin{array}{l} a_1^{(2)} = 0.00 \\ a_2^{(2)} = 0.40 \\ a_3^{(2)} = 0.25 \\ a_1^{(3)} = 0.91 \end{array}$$

6. Tensor Flow



7. Clasificación multietiqueta

Uno vs Todos: Múltiples unidades de salida



$$h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Peatón

$$h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Coche

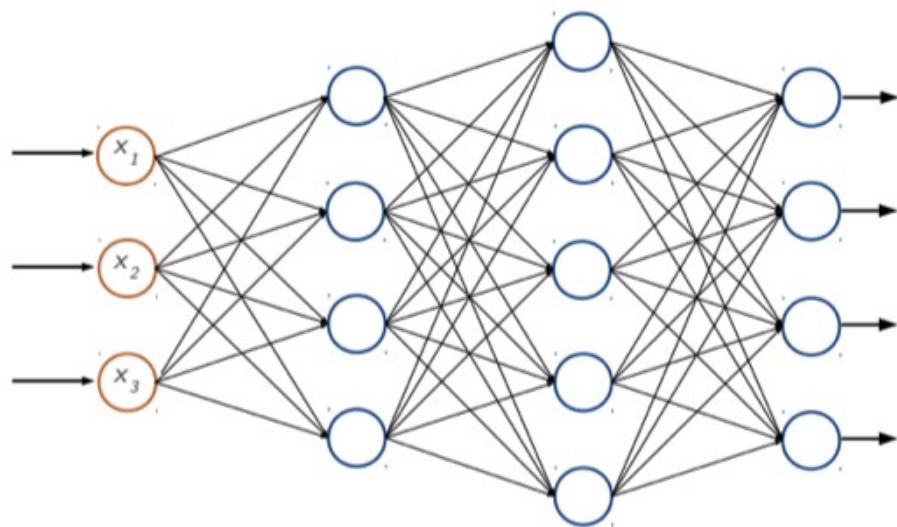
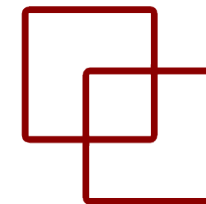
$$h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Moto

$$h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Camión

8. Aprendizaje en RNs



L = número de capas en la red.

s_l = número de unidades (sin la unidad de sesgo) en la capa l

Clasificación binaria:

- ✓ Una unidad de salida
- ✓ $y \in \{0, 1\}$

Clasificación multiclase:

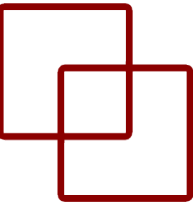
- ✓ K unidades de salida
- ✓ $y \in \mathbb{R}^k$

9. scikit-learn

- Contiene dos clases principales para RNAs
 - *neural_network.MLPClassifier*.
 - *neural_network.MLPRegressor*.
- Ambas comparten varios parámetros:
 - *alpha*. Parámetro que rige la regularización.
 - *activation*. Función de activación:
 - ‘*identity*’ (tenemos la $z=a$, no hay función de activación), ‘*logistic*’, ‘*tanh*’ o ‘*relu*’ (es lineal)
 - *hidden_layer_sizes*. Tupla que contiene el número de neuronas en cada capa intermedia.
 - *solver*. Algoritmo de optimización:
 - ‘*lbfgs*’, ‘*sgd*’ (gradiente descendiente estocástico) o ‘*adam*’



Gradiente Descendiente

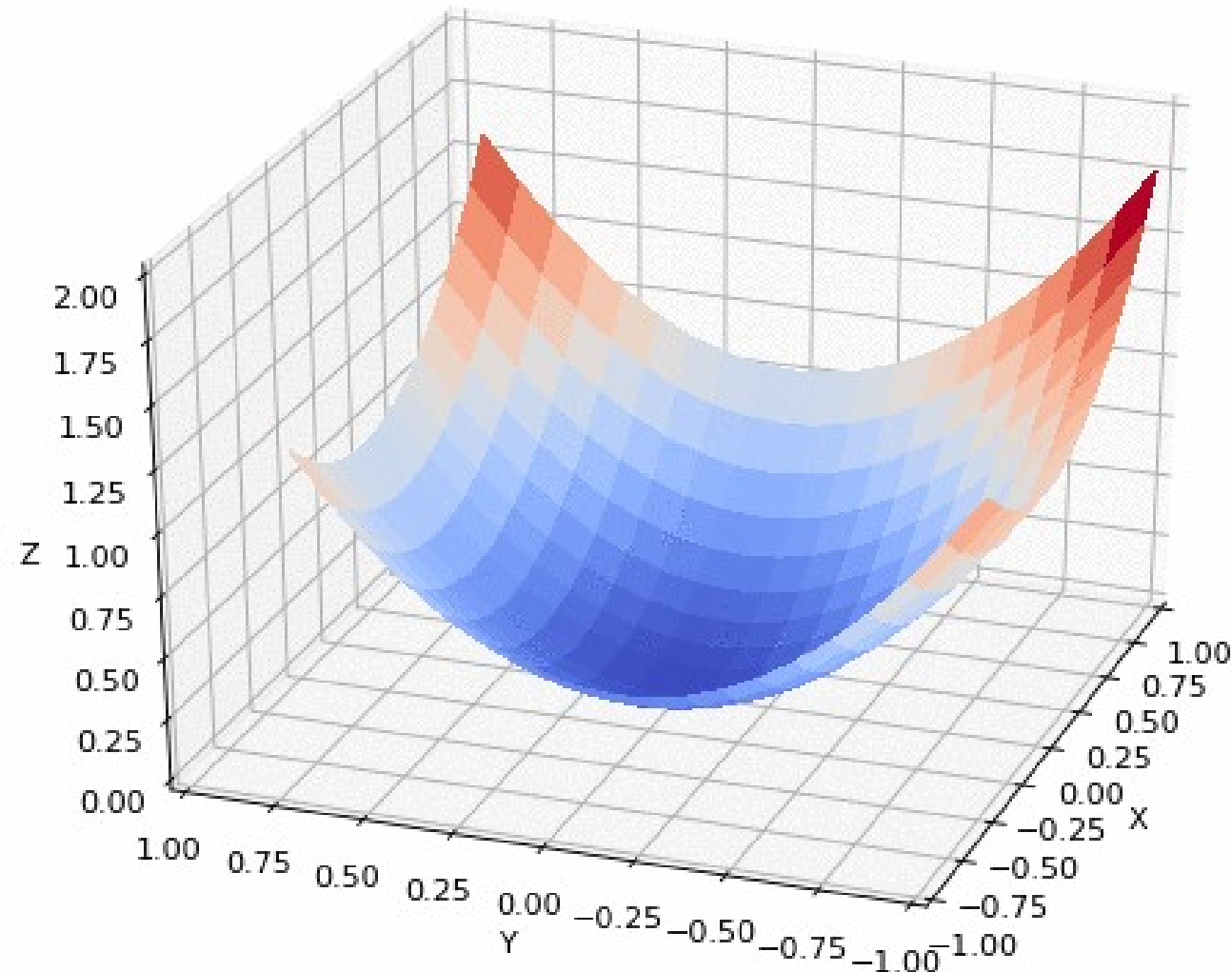
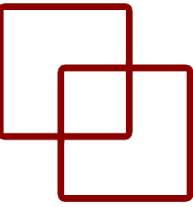


1. Contexto

- Dado que el coste representa lo alejado que estamos del resultado deseado, queremos **minimizar** dicho valor y, por tanto, alcanzar su valor mínimo (idealmente 0):
 - Usamos Gradiente Descendiente
 - **Hipótesis:** el valor a reducir decrece rápidamente si en cada parámetro de la fórmula, aplicamos un corrector negativo.
 - Este corrector o gradiente se obtiene a partir de la derivada parcial en el peso a establecer computado.
- La aplicación de cada gradiente (al completo) sobre cada peso no es una buena idea, ya que aumenta la posibilidad de caer en **óptimos locales**.
 - En su lugar, se aplica un **ratio de aprendizaje** de forma que en cada iteración, el valor de un eso se actualiza:

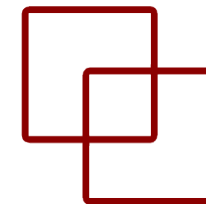
$$wi_{t+1} = wi_t * (1 - ratio) - ratio * gradient$$

2. Ejemplo



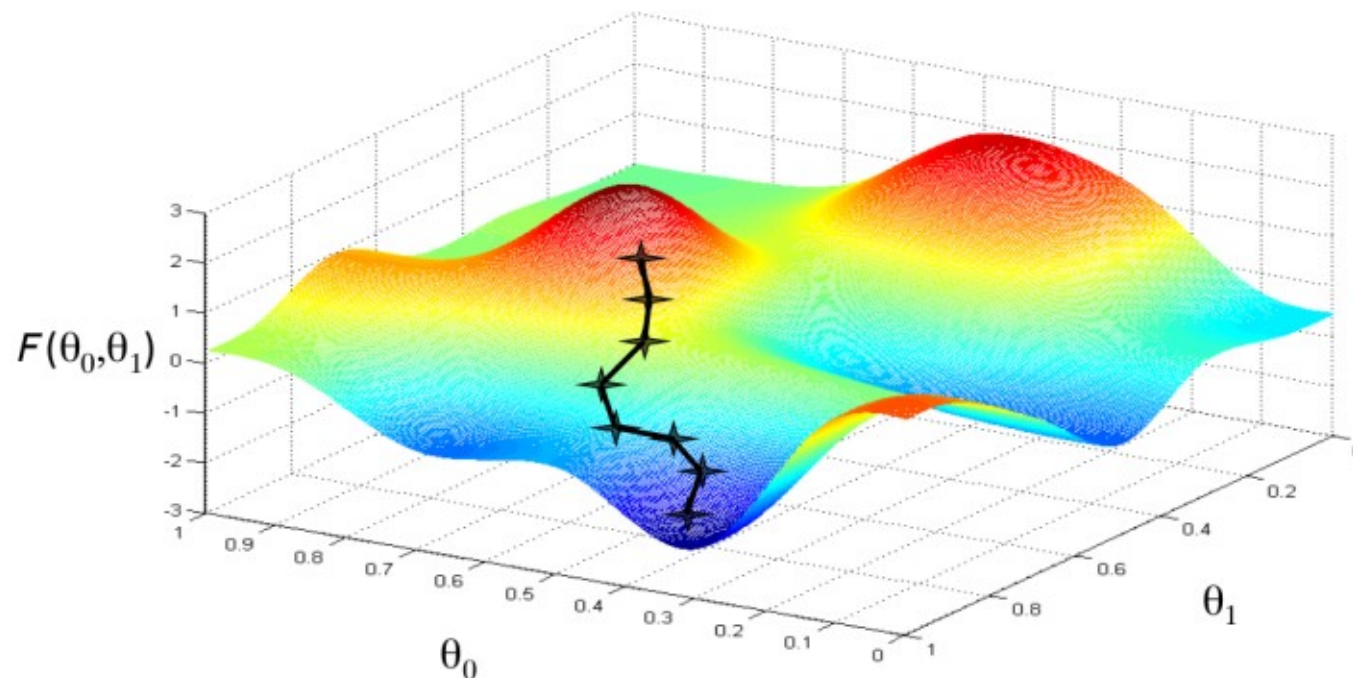
<https://medium.com/@pytholabs/multivariate-linear-regression-from-scratch-in-python-5c4f219be6a>

3. El gradiente descendiente (I)

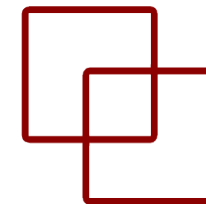


El método gradiente descendiente permite minimizar una función. Para ello, básicamente parte de unos valores para los parámetros elegidos aleatoriamente, y avanza en pasos sucesivos hacia un mínimo.

- Utilizando como ejemplo una función cualquiera: $F(\theta_0, \theta_1)$.
 - Se pretende minimizar $F(\theta_0, \theta_1)$
 - Se parte de unos valores θ_0 y θ_1 elegidos aleatoriamente.
 - Se actualizan θ_0 y θ_1 mientras se reduzca $F(\theta_0, \theta_1)$



3. El gradiente descendiente (II)



Algoritmo gradiente descendiente

inicializar aleatoriamente θ_0, θ_1

repetir hasta convergencia {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} F(\theta_0, \theta_1) \quad (\text{para } j = 0 \text{ y } j = 1, \alpha > 0)$$

}

Actualización simultánea: correcto

$$temp0 := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} F(\theta_0, \theta_1)$$

$$temp1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} F(\theta_0, \theta_1)$$

$$\theta_0 := temp0$$

$$\theta_1 := temp1$$

Incorrecto

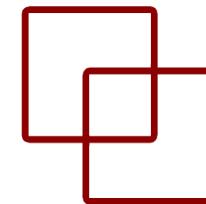
$$temp0 := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} F(\theta_0, \theta_1)$$

$$\theta_0 := temp0$$

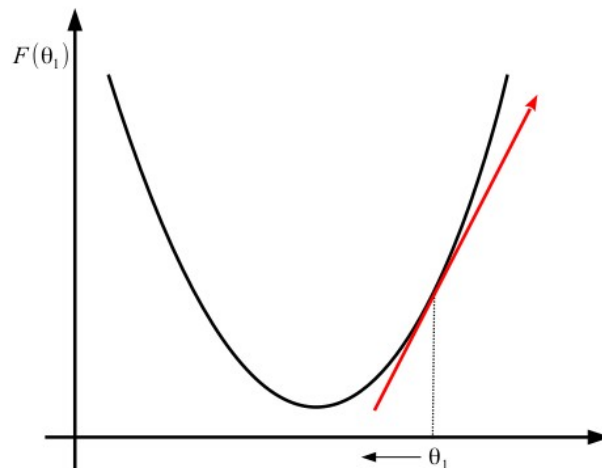
$$temp1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} F(\theta_0, \theta_1)$$

$$\theta_1 := temp1$$

3. El gradiente descendiente (III)



- Suponiendo una función cualquiera de **una** variable, $F(\theta_1)$
- El signo de la derivada nos dirá si θ_1 crece o decrece.
- Cuando la derivada de la tangente es 0 se estanca en un valor.

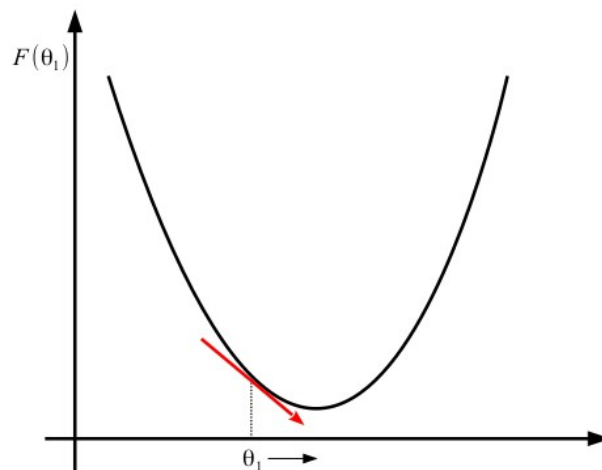


$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$$

$$\frac{d}{d\theta_1} F(\theta_1) \geq 0, \alpha > 0$$

$$\theta_1 := \theta_1 - \alpha \cdot (\text{número positivo})$$

θ_1 decrece



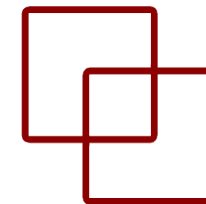
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$$

$$\frac{d}{d\theta_1} F(\theta_1) \leq 0, \alpha > 0$$

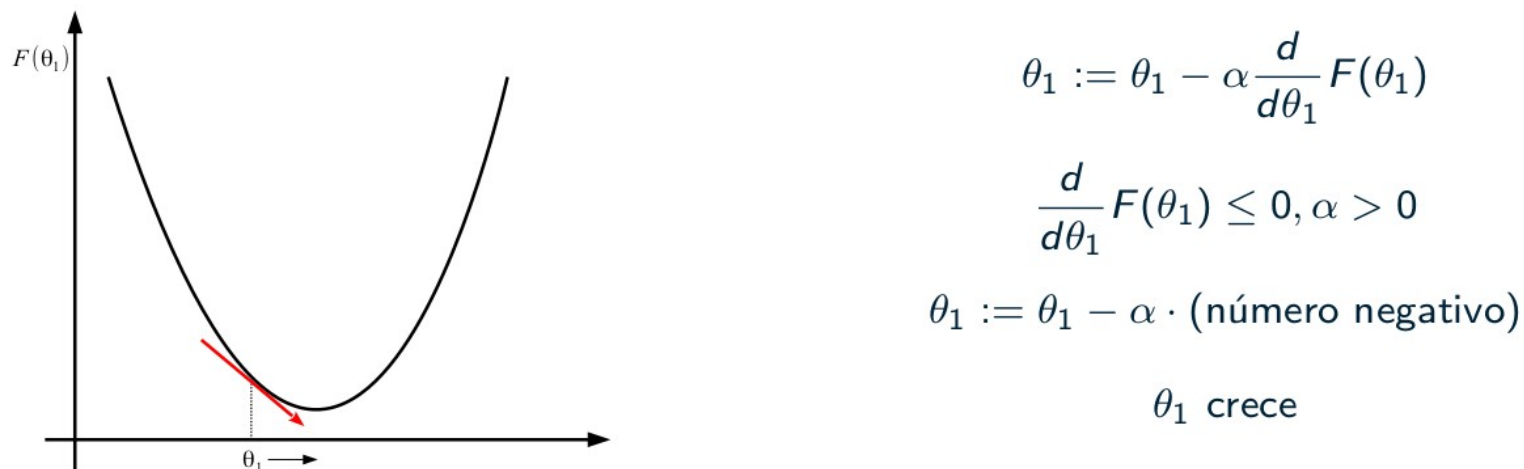
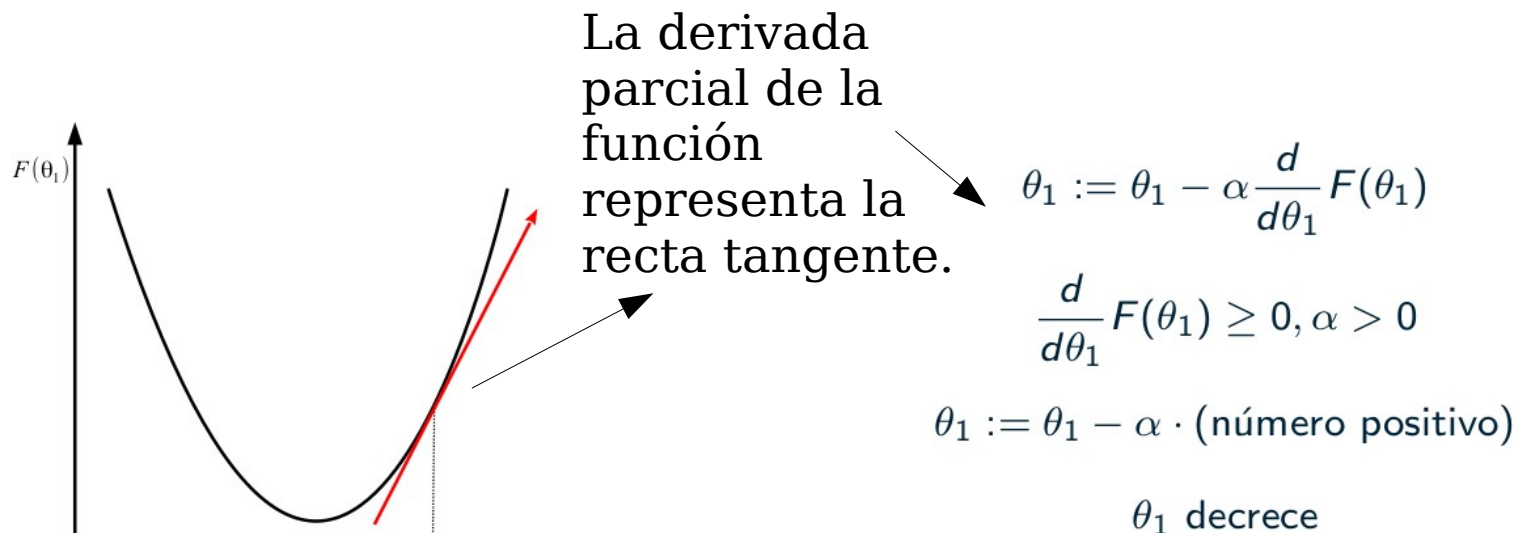
$$\theta_1 := \theta_1 - \alpha \cdot (\text{número negativo})$$

θ_1 crece

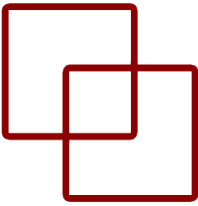
3. El gradiente descendiente (III)



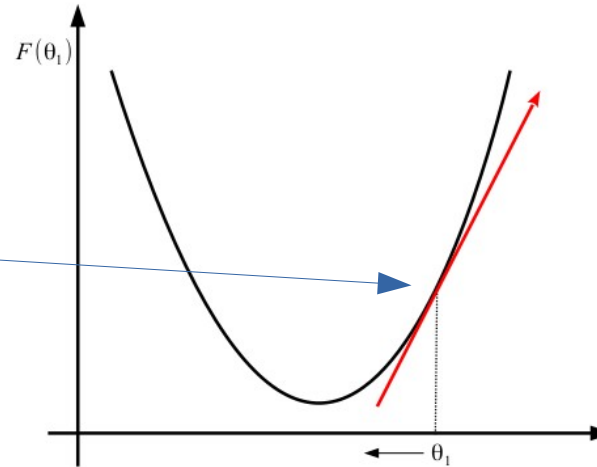
- Suponiendo una función cualquiera de **una** variable, $F(\theta_1)$
- El signo de la derivada nos dirá si θ_1 crece o decrece.
- Cuando la derivada de la tangente es 0 se estanca en un valor.



3. El gradiente descendiente (III)



Como la función crece la derivada es positiva

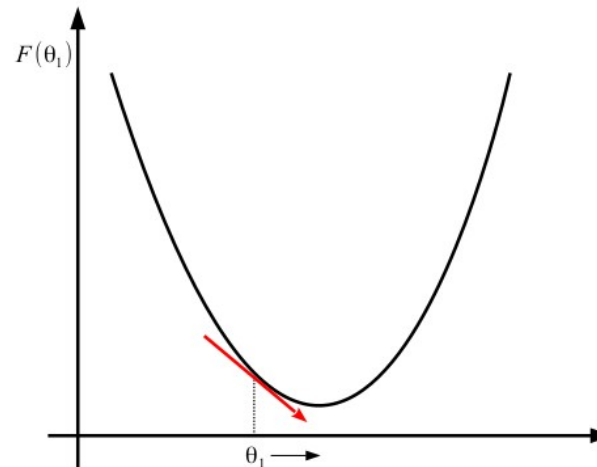


$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$$

$$\frac{d}{d\theta_1} F(\theta_1) \geq 0, \alpha > 0$$

$$\theta_1 := \theta_1 - \alpha \cdot (\text{número positivo})$$

θ_1 decrece



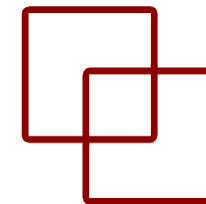
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$$

$$\frac{d}{d\theta_1} F(\theta_1) \leq 0, \alpha > 0$$

$$\theta_1 := \theta_1 - \alpha \cdot (\text{número negativo})$$

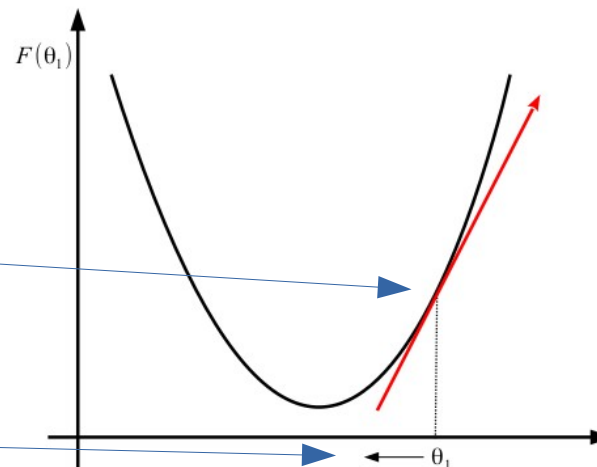
θ_1 crece

3. El gradiente descendiente (III)



Como la función crece la derivada es positiva

Por tanto, decrece y se acerca al punto de convergencia

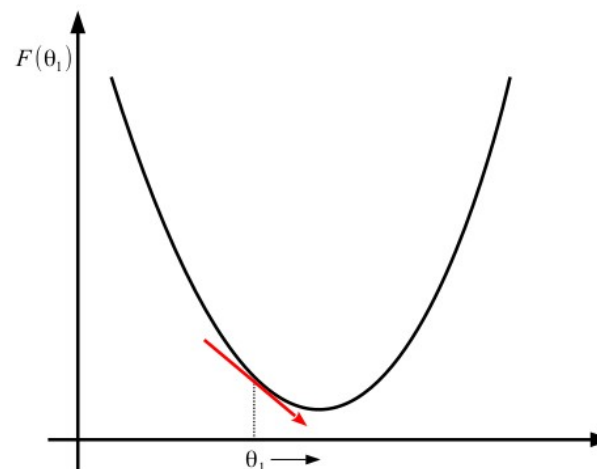


$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$$

$$\frac{d}{d\theta_1} F(\theta_1) \geq 0, \alpha > 0$$

$$\theta_1 := \theta_1 - \alpha \cdot (\text{número positivo})$$

θ_1 decrece



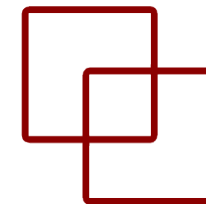
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$$

$$\frac{d}{d\theta_1} F(\theta_1) \leq 0, \alpha > 0$$

$$\theta_1 := \theta_1 - \alpha \cdot (\text{número negativo})$$

θ_1 crece

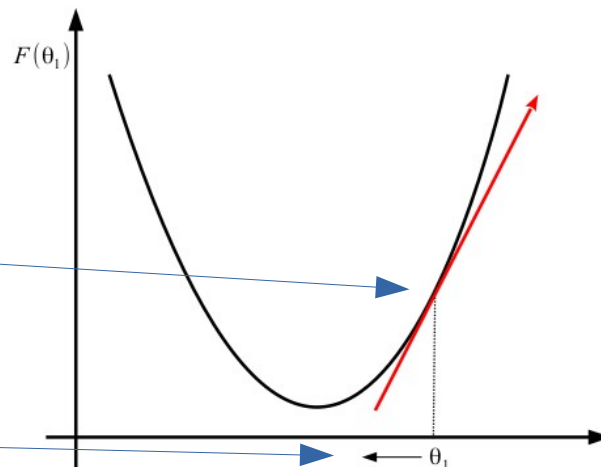
3. El gradiente descendiente (III)



Como la función crece la derivada es positiva

Por tanto, decrece y se acerca al punto de convergencia

Ahora la pendiente es negativa, porque la función decrece. Ahora la derivada es negativa y al restar un número negativo con $-\alpha$ entonces θ_1 crecerá y será positivo (se desplaza a la derecha)

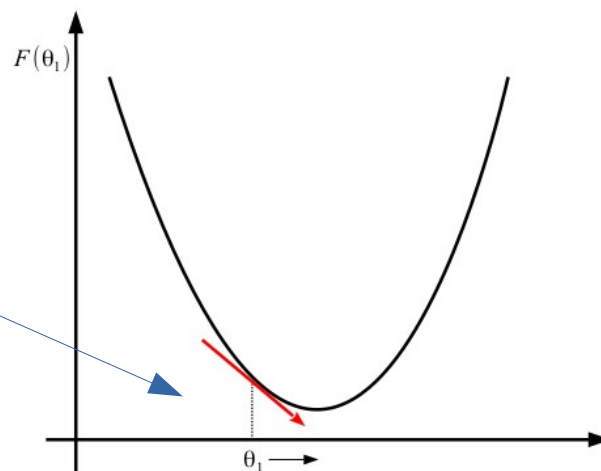


$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$$

$$\frac{d}{d\theta_1} F(\theta_1) \geq 0, \alpha > 0$$

$$\theta_1 := \theta_1 - \alpha \cdot (\text{número positivo})$$

θ_1 decrece



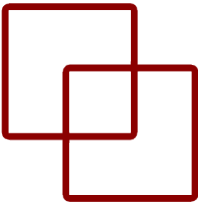
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$$

$$\frac{d}{d\theta_1} F(\theta_1) \leq 0, \alpha > 0$$

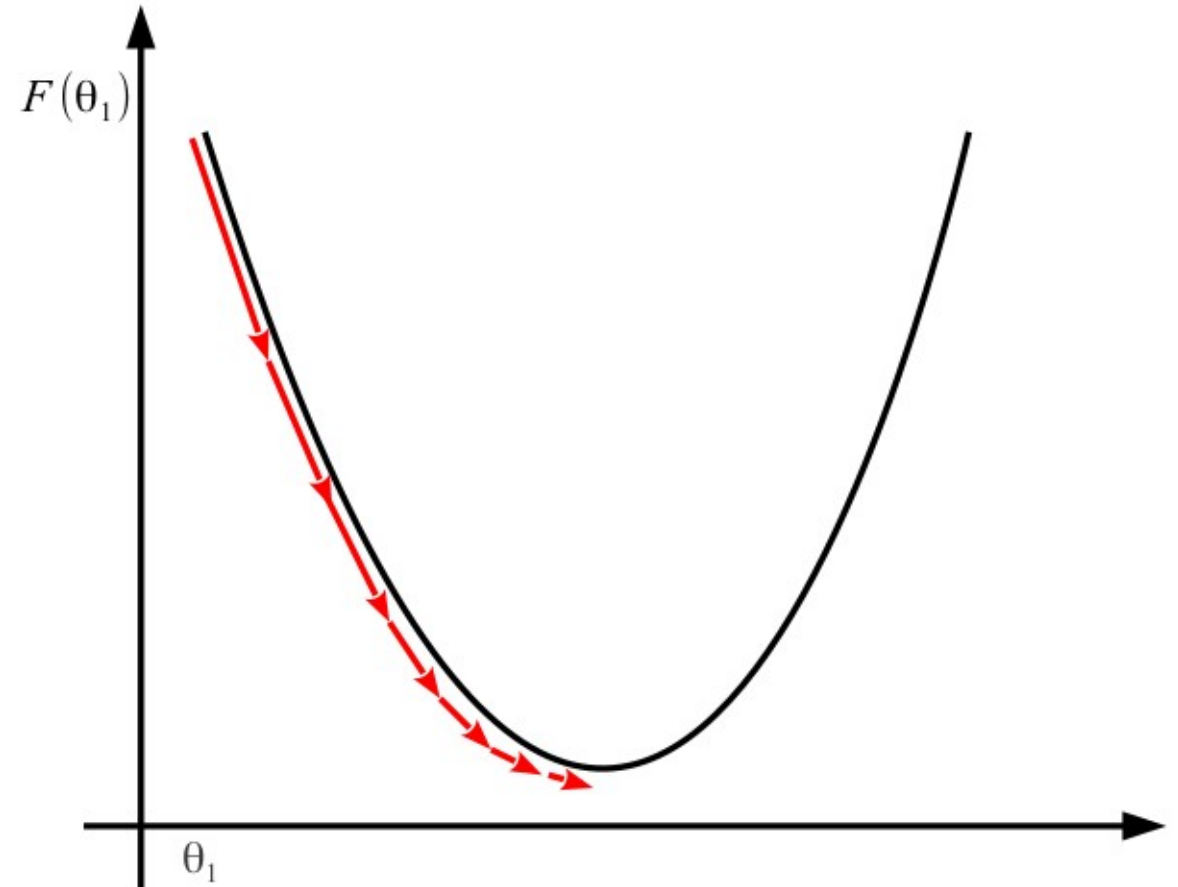
$$\theta_1 := \theta_1 - \alpha \cdot (\text{número negativo})$$

θ_1 crece

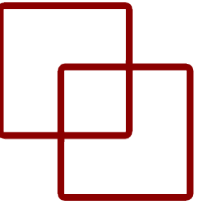
3. El gradiente descendiente (IV)



- Cuando el valor de los parámetros se aproxima a un mínimo, el algoritmo da pasos más pequeños de manera automática, ya que $\frac{d}{d\theta_1} F(\theta_1)$ es menor.
- Por tanto, no es necesario decrementar α a lo largo de las iteraciones. Debemos fijar un valor aceptable.
- Repasar lo de NumPy.

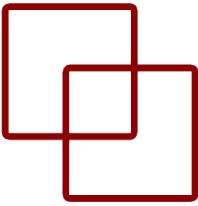


3. El gradiente descendiente (V)



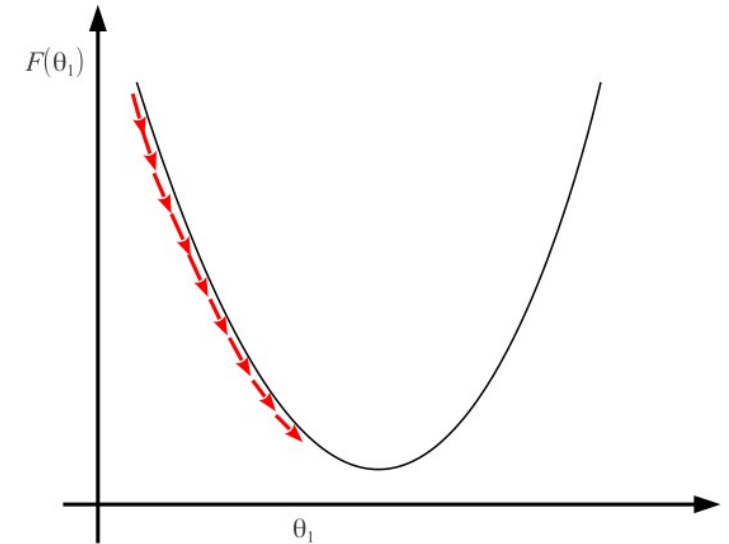
El valor α se denomina razón de aprendizaje. $\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$

3. El gradiente descendiente (V)

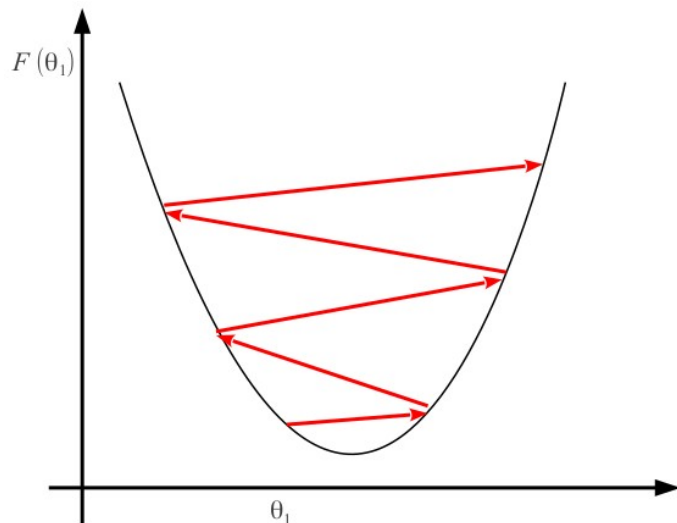


El valor α se denomina razón de aprendizaje. $\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} F(\theta_1)$

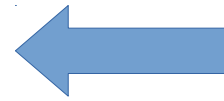
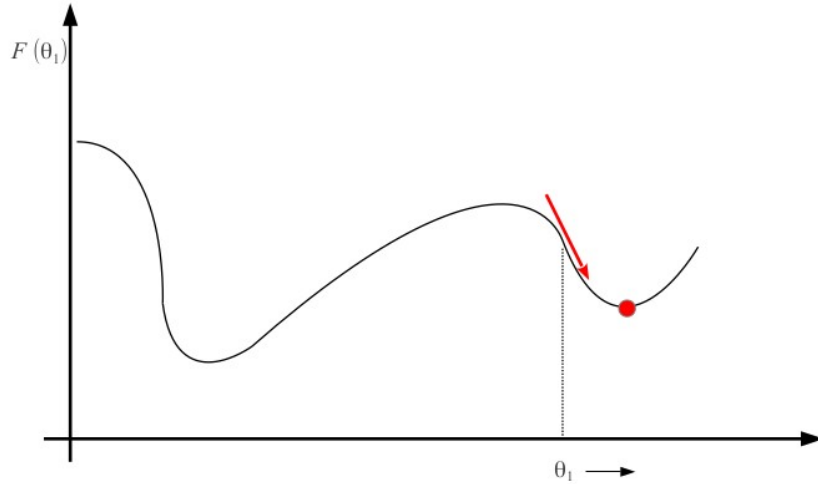
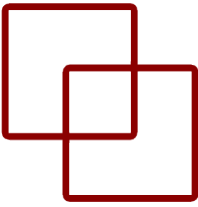
Si α es demasiado pequeña, la convergencia puede ser demasiado lenta.



Si α es demasiado grande, el algoritmo puede no converger, e incluso divergir.

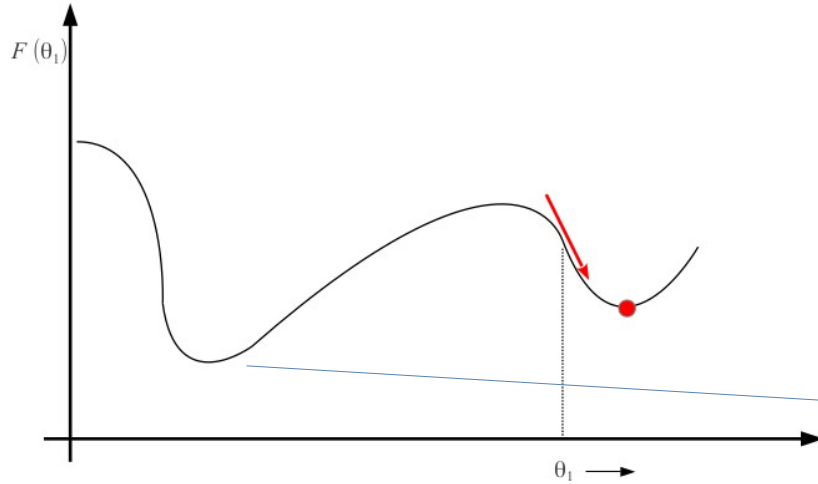
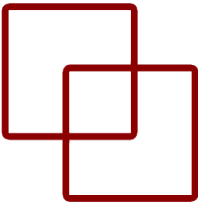


3. El gradiente descendiente (VI)



El método gradiente descendiente siempre converge a un óptimo local. En algunos casos, este óptimo coincide con el global o no.

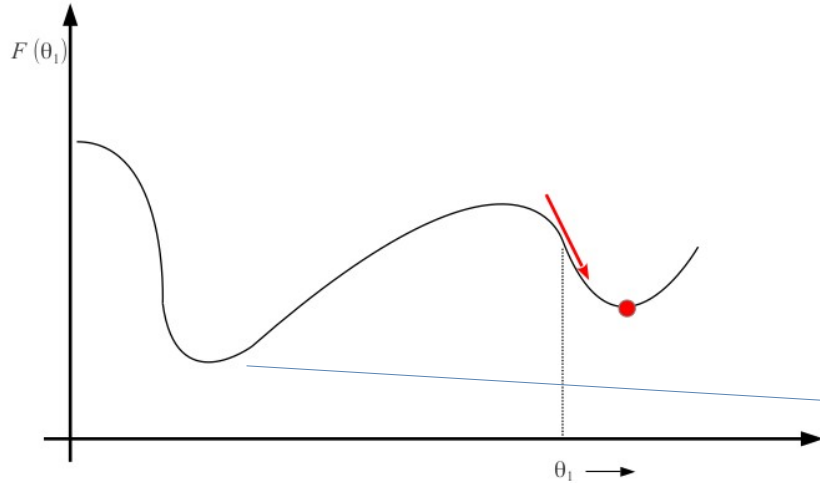
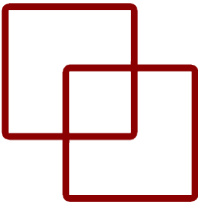
3. El gradiente descendiente (VI)



El método gradiente descendiente siempre converge a un óptimo local. En algunos casos, este óptimo coincide con el global o no.

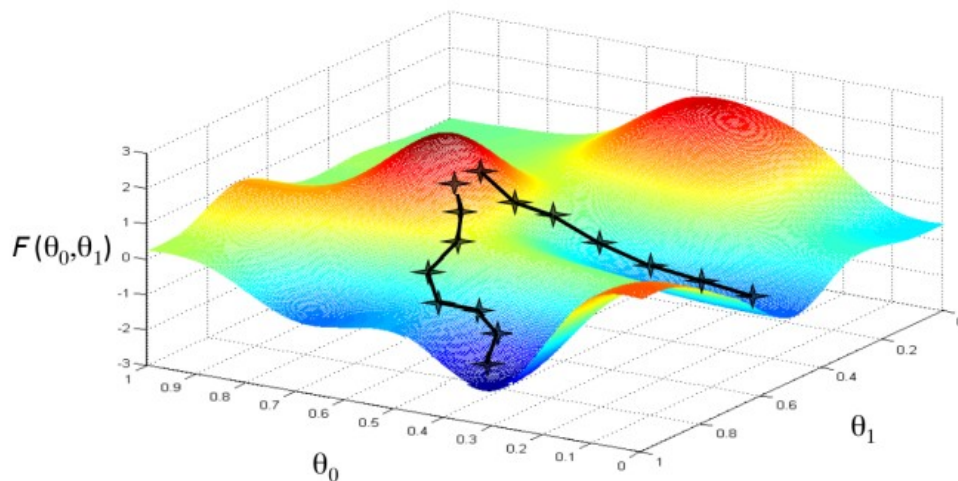
Fijarse que hay otro óptimo mínimo.

3. El gradiente descendiente (VI)



El método gradiente descendiente siempre converge a un óptimo local. En algunos casos, este óptimo coincide con el global.

Fijarse que hay otro óptimo mínimo.

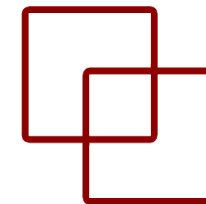


El punto al que converge el algoritmo depende de los valores iniciales de los parámetros.

- Aquí tenemos dos puntos de partida que son diferentes en el mínimo de convergencia

4. Gradiente Descendiente (I)

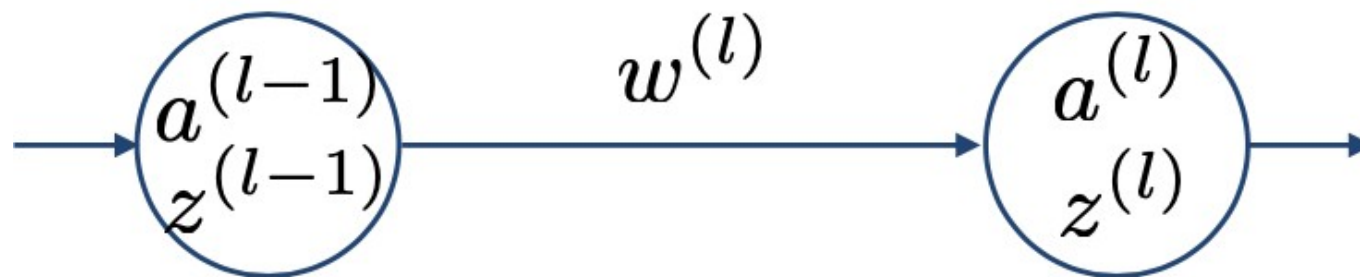
Supongamos la RN más sencilla posible



$$E = (a^{(l)} - t)^2$$

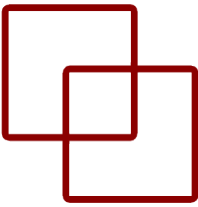
$$a^{(l)} = f(z^{(l)})$$

$$z^{(l)} = a^{(l-1)} \times w^{(l)} + b^{(l)}$$



4. Gradiente Descendiente (II)

Supongamos la RN más sencilla posible



- El **descenso del gradiente** (o regla delta, LMS, adaline, Widrow-Hoff), consiste en dirigir la búsqueda hacia el descenso por máximo gradiente en el espacio de hipótesis de posibles pesos.

- Consideraremos el error:

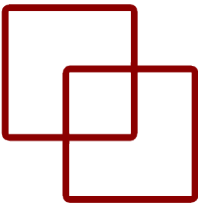
$$E(\mathbf{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

- Con t_d el valor real asociado a la variable de respuesta para el caso d -ésimo y;
 - o_d el valor de salida producido por la red para dicho caso.
- Para comenzar supongamos una función lineal asociada a las neuronas:

$$o(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} = \sum_i w_i \cdot x_i$$

4. Gradiente Descendiente (III)

Supongamos la RN más sencilla posible



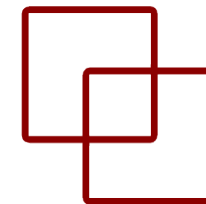
- Para calcular la dirección de máximo descenso sobre la superficie de error calculamos la derivada de E con respecto a todos los pesos:

$$\nabla E(\mathbf{w}) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right]$$

- El gradiente $\nabla E(\mathbf{w})$ representa la dirección que produce el mayor aumento de E desde un determinado punto del espacio.
 - Su negación $-\nabla E(\mathbf{w})$ representa, por tanto, la dirección que maximiza la disminución del error.
- Así, la **regla delta** de actualización queda: $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$ con $\Delta \mathbf{w} = -\eta \nabla E(\mathbf{w})$
- Para cada componente: $w_i \leftarrow w_i + \Delta w_i$ con $\Delta w_i = -\eta \nabla E(w_i)$
- η es la **razón de aprendizaje** y debe ser pequeño (0.05, 0.1,...)

4. Gradiente Descendiente (IV)

Supongamos la RN más sencilla posible



- En el caso general con funciones de activación continuas y diferenciables tenemos:

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d) f'(o_d) x_{id}$$

- Por ejemplo:

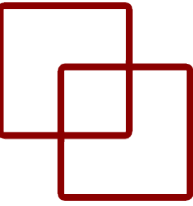
- Logística: $f(x) = \frac{1}{1+e^{-x}} \implies f'(x) = f(x)(1 - f(x))$
- Tanh: $f(x) = \tanh(x) \implies f'(x) = (1 - f(x)^2)$



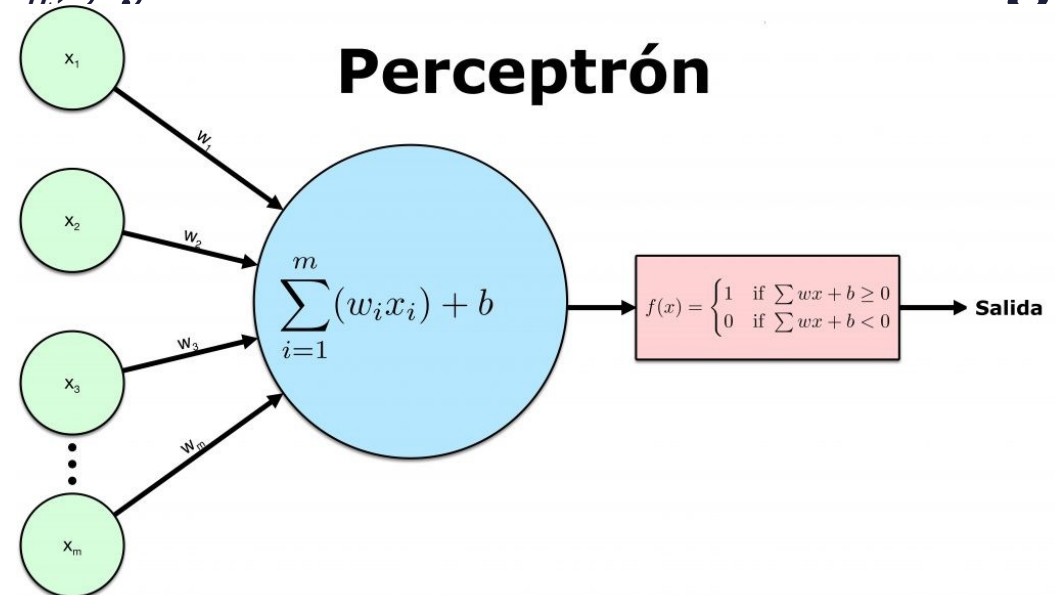
LAB CTIC  UNI

Perceptron Multicapa / Redes Neuronales

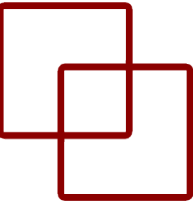
1. Perceptron multicapa



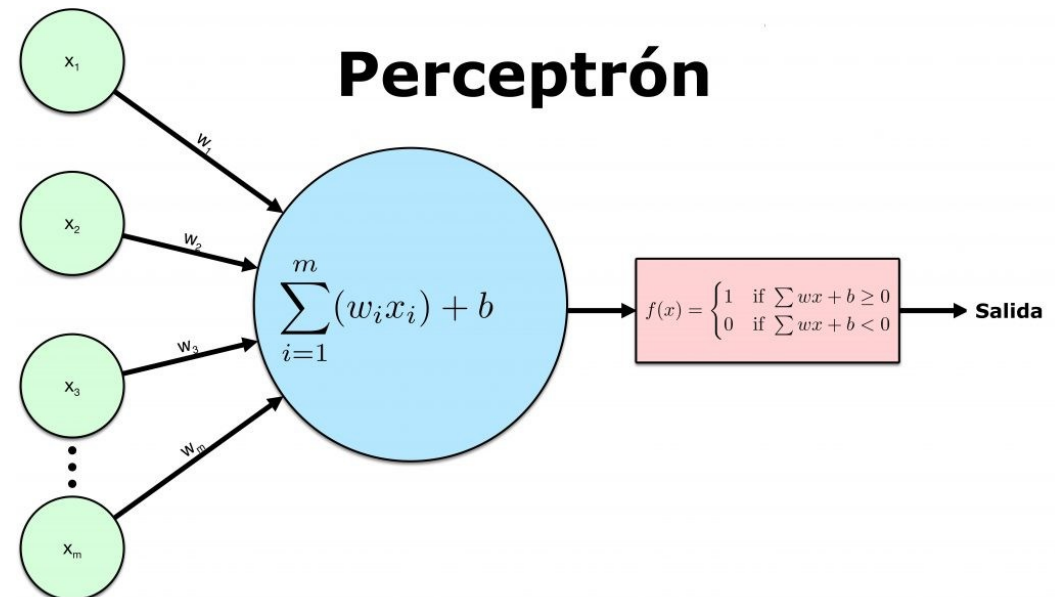
- Perceptrón multicapa parte de un modelo simple desarrollado en la década de los 50: **Perceptrón**
- Este modelo toma una serie de N **entradas** ($x_1, \dots, x_n$) y genera una salida a partir de una función que utiliza las entradas, una serie de **pesos** ($w_1, \dots, w_n$), y un factor adicional: **sesgo** (b)



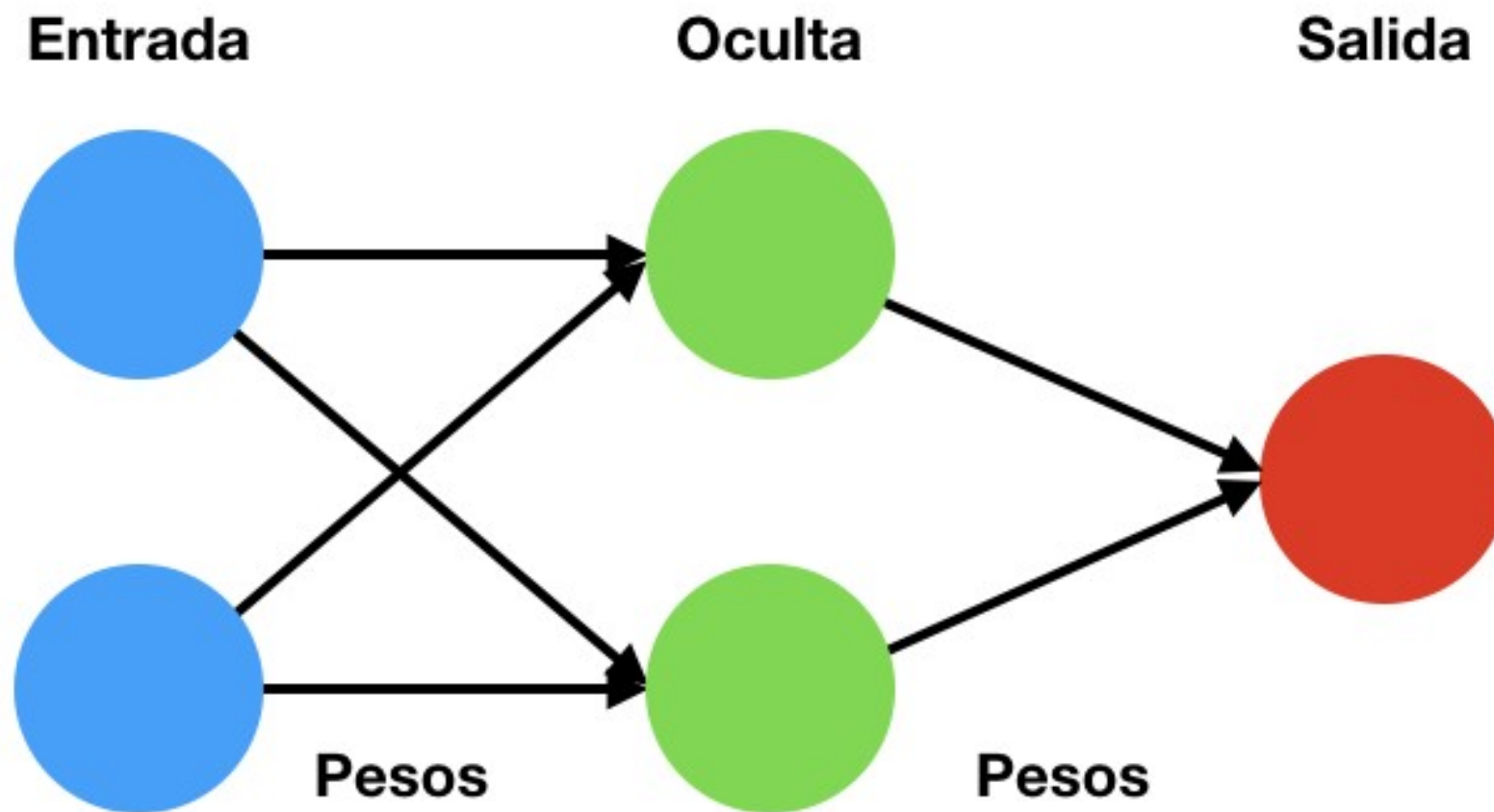
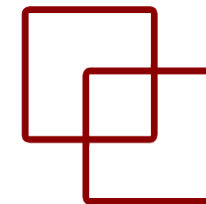
2. Conceptos clave



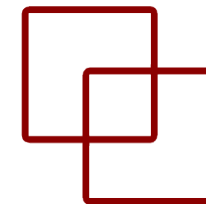
- Cómo se combinan las entradas → pesos (w)
- Cómo se determina la salida → función de activación
 - La función de activación suele establecerse como:
$$\begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$
- Cómo se establecen estos parámetros → aprendizaje
- Qué otros parámetros intervienen → sesgo



3. Arquitectura



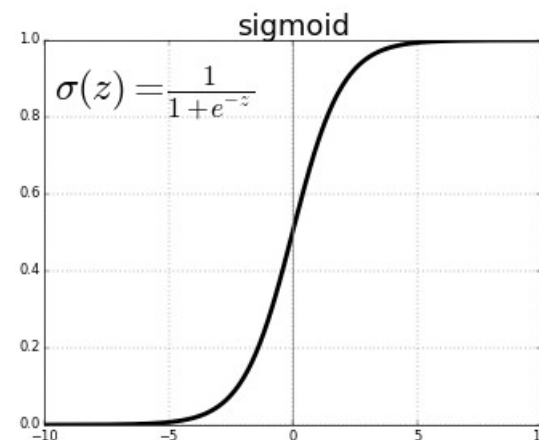
4. Capas



- El MLP se entiende como la evolución del Perceptron Simple donde aparecen diferentes capas intermedias (llamadas **ocultas**) entre la entrada y salida
- En estas capas intermedias podemos reemplazar la función anterior por una **sigmoidal***

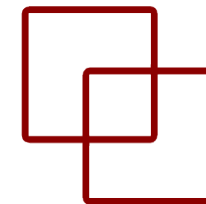
$$z = b + \sum_i w_i x_i$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



* Algunos autores limitan el uso del término MLP al uso de funciones básicas y al introducir sigmoidales pasan al término de Redes Neuronales

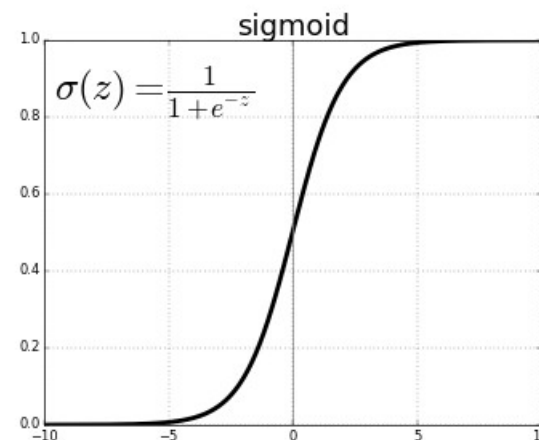
5. Función sigmoideal

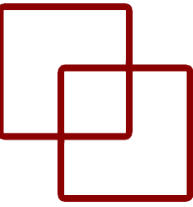


- El uso de esta función introduce ciertos cambios:
 - La generación de **salidas** que ya no son **binarias** sino en el rango $[0,1]$
 - La capacidad de resolución de **problemas no lineales**

$$z = b + \sum_i w_i x_i$$

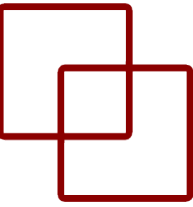
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$





6. Características

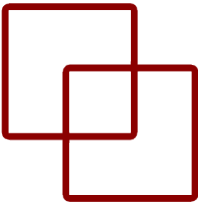
- El número de capas ocultas aumenta la complejidad de la red
 - **Afecta** al proceso de clasificación, pero sobre todo **aumenta** la complejidad del aprendizaje
- El proceso de aprendizaje puede verse como la búsqueda de reducir un valor llamado **Coste**.
- Este valor Coste representa lo lejos que está nuestra red de producir un **resultado perfecto** sobre un conjunto de entrenamiento.



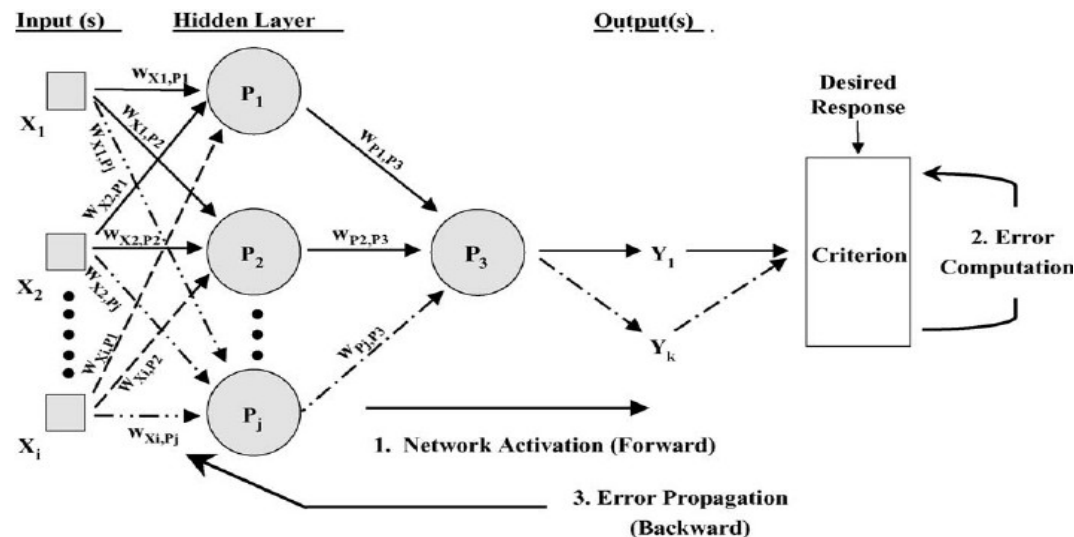
7. Aprendizaje

- El **aprendizaje** es clave → computación del **coste**: clave
- En lugar de computar el error absoluto, se buscan **funciones de coste** que varíen ligeramente ante pequeños cambios en los parámetros (pesos o sesgo).
- En este proceso se hace uso de dos técnicas:
 - **Backpropagation**
 - **Gradiente Descendiente**
- El aprendizaje es un proceso iterativo donde se introduce un parámetro:
 - Factor o Ratio de Aprendizaje
 - Factores altos aumentan la velocidad, pero también el **riesgo** de caer en óptimos locales.

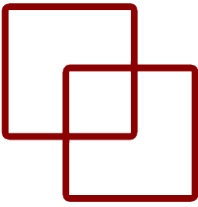
8. Proceso de Aprendizaje (I)



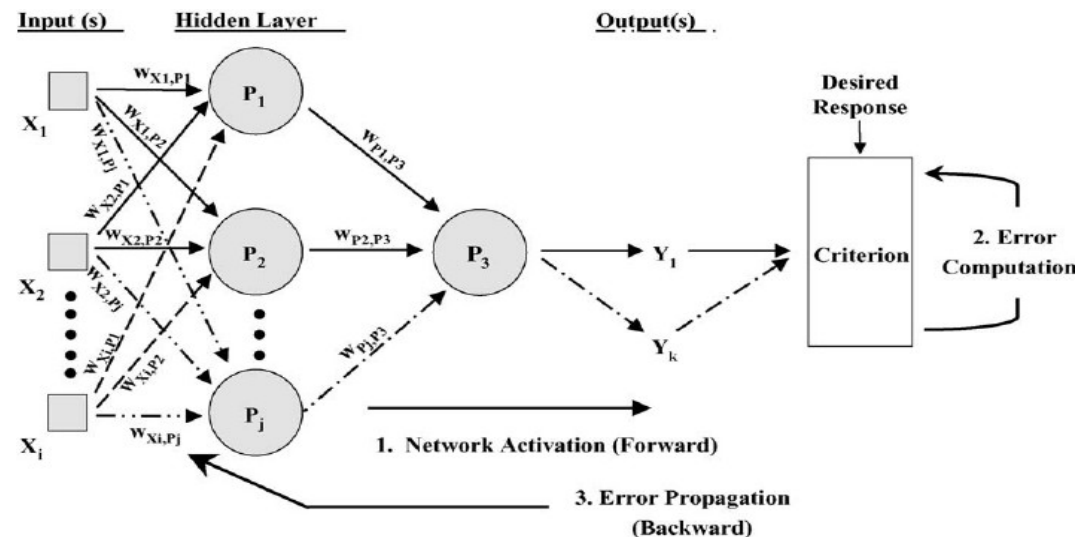
- Consta de dos etapas **forward** y **backward**, partiendo de una red con pesos inicializados (normalmente de forma aleatoria) y un conjunto de entrenamiento con valores de entrada y salida esperada.
- **Forward**: a partir de unos valores de entrada, se realizan los **cálculos intermedios**, se almacenan para cálculos posteriores, y se obtiene un resultado.
 - Se compara con el **esperado** y se computa el **error**

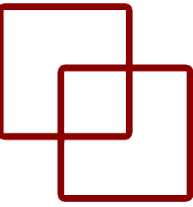


8. Proceso de Aprendizaje (II)



- **Backward:** Se aplica la regla de la cadena de forma que en cada neurona, se actualicen sus parámetros asociados (pesos) en base a
 - 1) El ratio de aprendizaje: Suele ser una constante (normalmente 0.05)
 - 2) El gradiente computado para esta neurona: Es la **derivada parcial** del error total (descomponiendo la fórmula) con respecto al peso de la neurona a actualizar.





9. Estimar el error

- Para una red con N neuronas de salida, podemos estimar el error usando el error cuadrático medio.

$$E_{total} = \sum \frac{1}{2} (target - output)^2$$

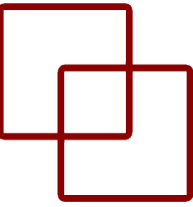
... pero luego veremos que otras fórmulas de error son más apropiadas

- Más adelante profundizaremos en la formulación y los conceptos de gradiente descendiente y *Backpropagation*



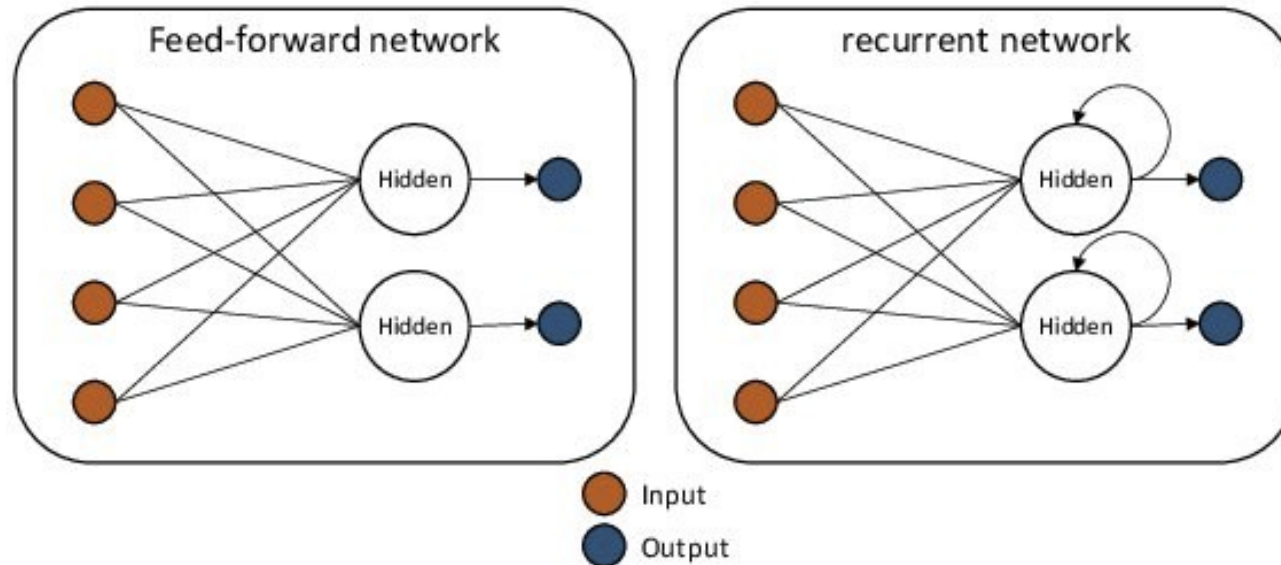
LAB CTIC  UNI

Redes Feed Forward y Recurrent Neural Networks

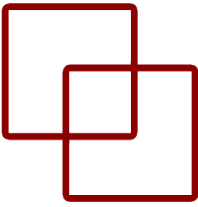


1. Feed Forward

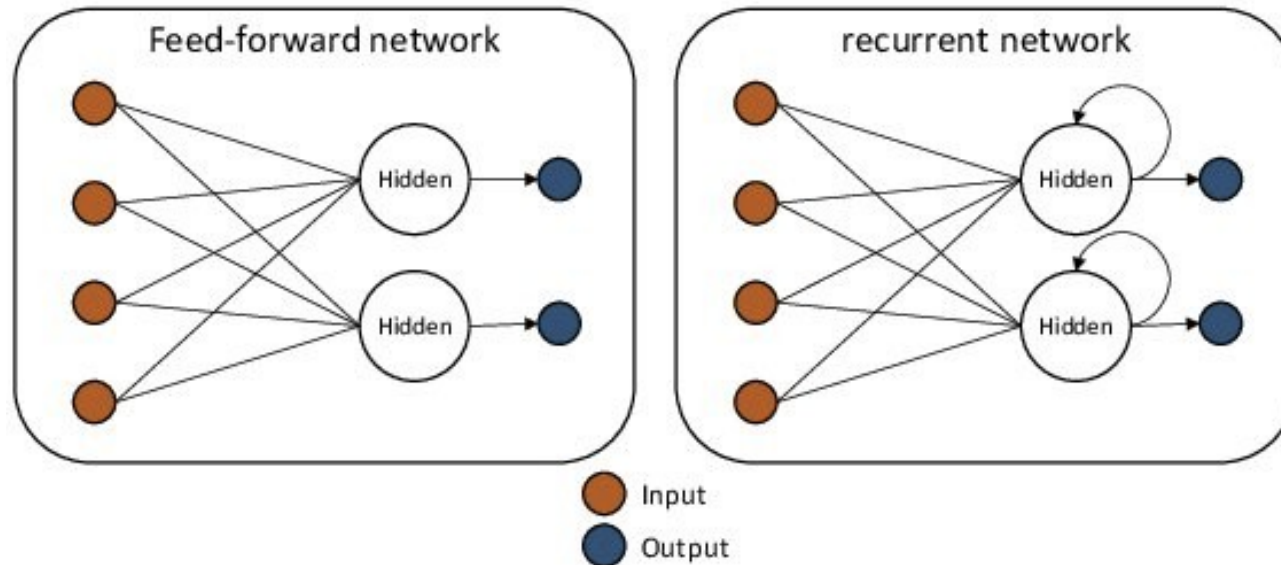
- A pesar de que se verán en la última sesión, es importante conocer la diferencia entre las dos principales **topologías** de redes
- Las llamadas **Feed Forward** se corresponden con el modelo tradicional, y se definen como **redes sin ciclos** ni **bucles** durante la clasificación.
 - El flujo de datos pasa de las capas de entrada a las intermedias, y finalmente a la capa de salida donde se obtiene el resultado final

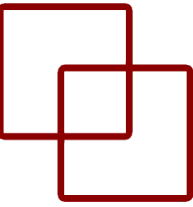


2. Recurrent Neural Networks



- Las redes Recurrent Neural Networks (RNNs), por el contrario, permiten la **aparición de bucles** en su arquitectura.
- Estos bucles tienen diferentes implicaciones, como la capacidad de aprovechar la temporalidad de una secuencia de entrada a través de un **estado**
 - Podemos decir que una red puede procesar una imagen en el instante t , y utilizar este último estado para procesar la siguiente imagen capturada en el instante $t+1$.



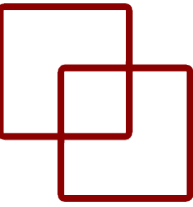


3. Características de FFN y RNN

- Los resultados usando RNNs han sido especialmente buenos hasta el momento en el reconocimiento del audio
 - Línea de investigación que permanece abierta
- Sin embargo, son **difíciles de entrenar** y en muchos casos no resulta práctico su uso ante problemas que puedan resolverse con redes Feed Forward.
 - Recomendable únicamente para problemas que requieran esa potencia.
- Existe un tercer tipo de red con enlaces no dirigidos llamadas **Symmetric o Restricted Boltzmann Machines**.

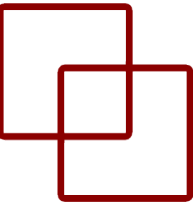


Pesos, parámetros sesgo y activación



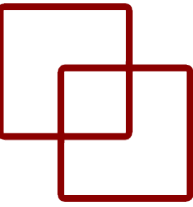
1. Contexto

- Por tanto, el **resultado**, a nivel de neurona, se computa como:
 - Valores de entrada
 - Pesos asociados
 - Parámetro de sesgo
- Entonces, para una neurona conectada con n neuronas de una capa anterior tendremos:
 - **n pesos**, pero
 - Un **único** parámetro de **sesgo** (b)



2. Aprendizaje y activación

- Junto a su arquitectura, los **pesos y sesgo** codifican el conocimiento aprendido por una red neuronal:
 - El proceso de aprendizaje tiene por objetivo el establecimiento de estos parámetros.
 - Para que este proceso fluya correctamente debemos prestar atención al **proceso de activación**
 - La **activación** de una neurona determina el valor computado que esta neurona propaga a las siguientes capas



3. Función de activación

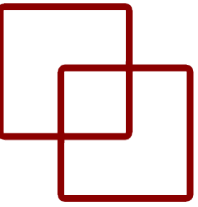
- La función de activación sirve como **abstracción** de qué neuronas **intervienen** en el proceso de clasificación, aunque tiene otros objetivos
 - Simplifica y orienta el proceso de aprendizaje

$$\begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

- Partiendo del modelo más sencillo (Perceptrón), vimos que las neuronas únicamente propagaban valores binarios, pero a continuación veremos **diversas funciones** (y sus implicaciones)

3. Función de activación

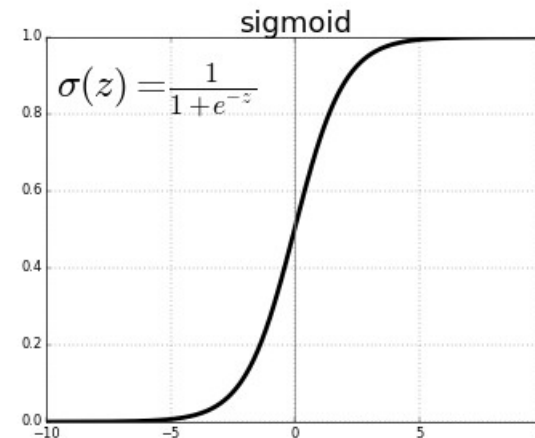
Sigmoidal (I)



- La función de activación usada en los perceptrones parte de la **limitación** de resolver **problemas lineales**.
- Además de esta función, se ha visto la **sigmoidal**, la cual aporta una mayor versatilidad para resolver problemas complejos y no solamente problemas lineales.

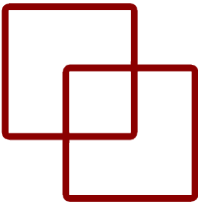
$$z = b + \sum_i w_i x_i$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



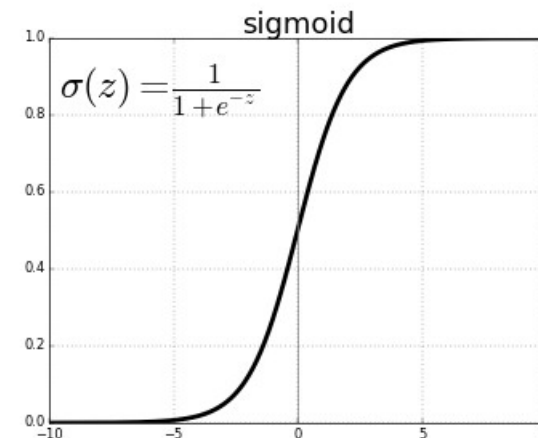
3. Función de activación

Sigmoidal (II)



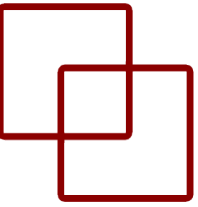
- Una de las principales ventajas de la función sigmoideal es la **acotación** de la salida en el rango $[0,1]$
- Sin embargo, esta función tiende a tener **pequeñas variaciones** ante los cambios en los parámetros de entrada

$$z = b + \sum_i w_i x_i$$
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



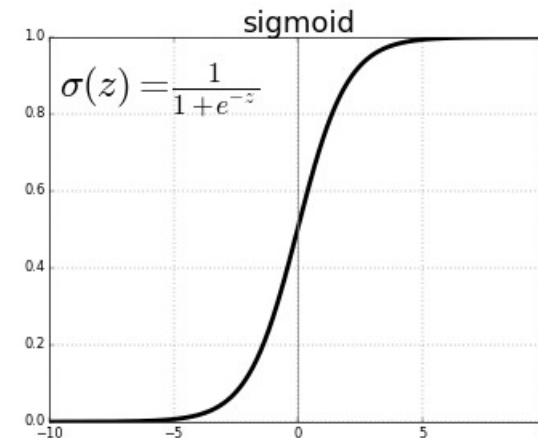
3. Función de activación

Sigmoidal (III)



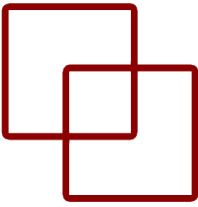
- Este hecho afecta al problema conocido como *vanishing gradient*, dificultando el proceso de aprendizaje.
 - ***Vanishing gradient***: durante el aprendizaje algunas variaciones en los parámetros no se traducen en diferencias lo suficientemente significativas como para tomarse en cuenta.
 - Parecido a problemas de optimización, si el cambio que hacemos no tiene grandes efectos en el resultado vamos a descartar el cambio.
- A pesar de estas limitaciones, la función sigmoidal sigue siendo ampliamente utilizada.

$$z = b + \sum_i w_i x_i$$
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

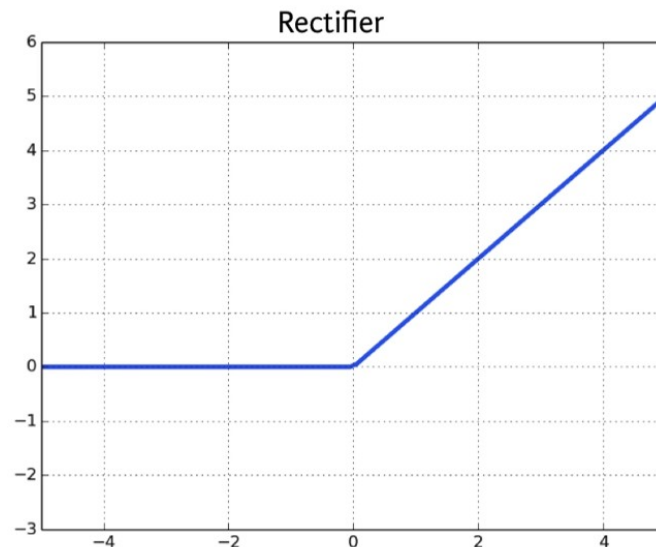


3. Función de activación

ReLu (I)



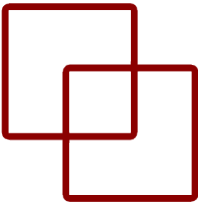
- La función de activación más usada en Deep Learning es la conocida como **ReLu** (*Rectified Linear Unit*).
 - Valor de entrada si es positivo, o 0 si es cero o negativo
- Puede pensarse que presenta las mismas restricciones que las funciones lineales... pero ReLu **no es lineal** y sus combinaciones tampoco lo son



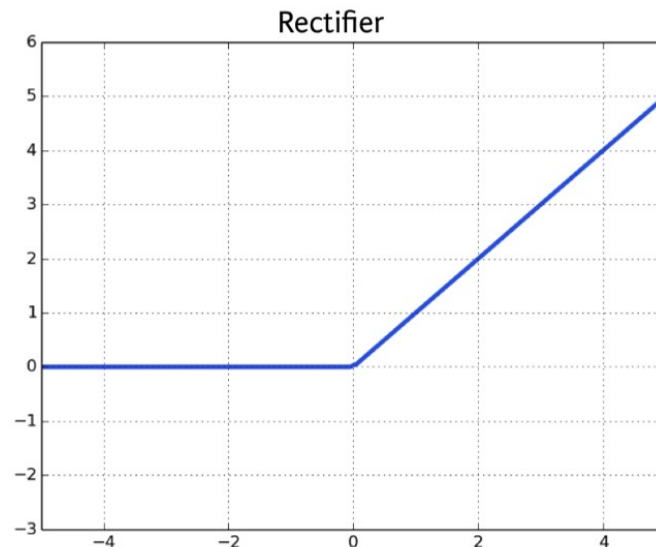
$$RELU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

3. Función de activación

ReLu (II)



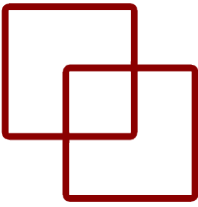
- La principal ventaja de ReLu es su **aportación al proceso de clasificación y aprendizaje**:
 - Funciones como la sigmoideal se traducen en clasificaciones donde la mayor parte de las neuronas intervienen en el proceso → **activaciones densas**.
 - Ineficiente: las clasificaciones sobre subconjuntos de datos de neuronas obtienen mejores resultados.
 - ReLu fuerza la no activación de las neuronas con valores negativos, lo que favorece **activaciones dispersas** → se obtiene el mismo resultado pero más rápido.



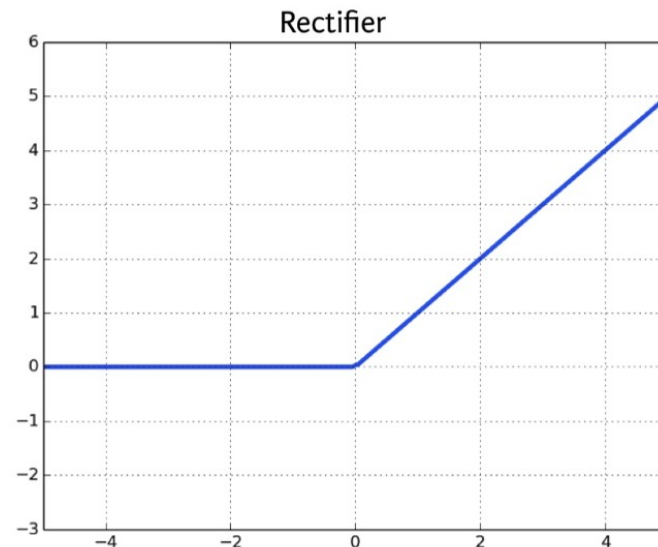
$$RELU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

3. Función de activación

ReLu (III)



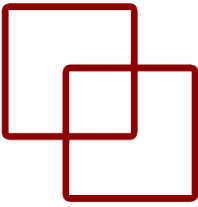
- A pesar de la simplicidad de ReLu, su introducción en las redes neuronales ha tenido un **enorme impacto**.
 - Algunos trabajos muestran cómo el aprendizaje se **acelera hasta 6 veces** al utilizar ReLu en lugar de la función sigmoidal o variaciones de la misma.
 - La computación también es más eficiente
- También posee algunas desventajas, como el llamado ***Dying ReLu problem***
 - Algunas neuronas dejan de aprender y quedan bloqueadas en un estado de no activación



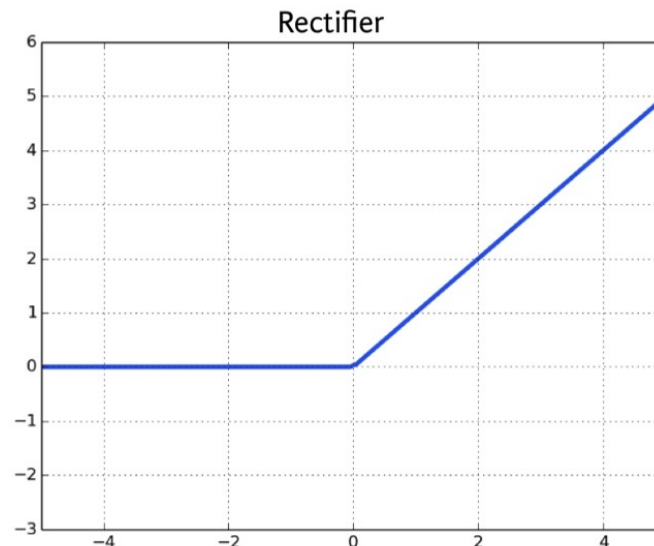
$$ReLU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

3. Función de activación

ReLu (III)



- A pesar de la simplicidad de ReLu, su introducción en las redes neuronales ha tenido un **enorme impacto**.
 - Algunos trabajos muestran cómo el aprendizaje se **acelera hasta 6 veces** al utilizar ReLu en lugar de la función sigmoidal o variaciones de la misma.
 - La computación también es más eficiente
- También posee algunas desventajas, como el llamado ***Dying ReLu problem***
 - Algunas neuronas dejan de aprender y quedan bloqueadas en un estado de no activación

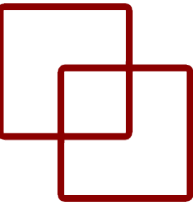


$$RELU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

Estas neuronas dejan de aprender

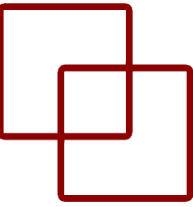


Softmax



1. Contexto

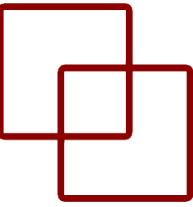
- La capa final de una red neuronal (*output*) se utiliza para **interpretar el resultado** generado durante el proceso de clasificación.
- Además de conocer la categoría o clase que debemos asociar a la entrada proporcionada, la interpretación se refiere a las **diferencias** entre valores obtenidos.
- Tanto si utilizamos la función Sigmoidal como ReLu, las neuronas de la capa final presentarán unos valores que no son válidos para una interpretación directa.



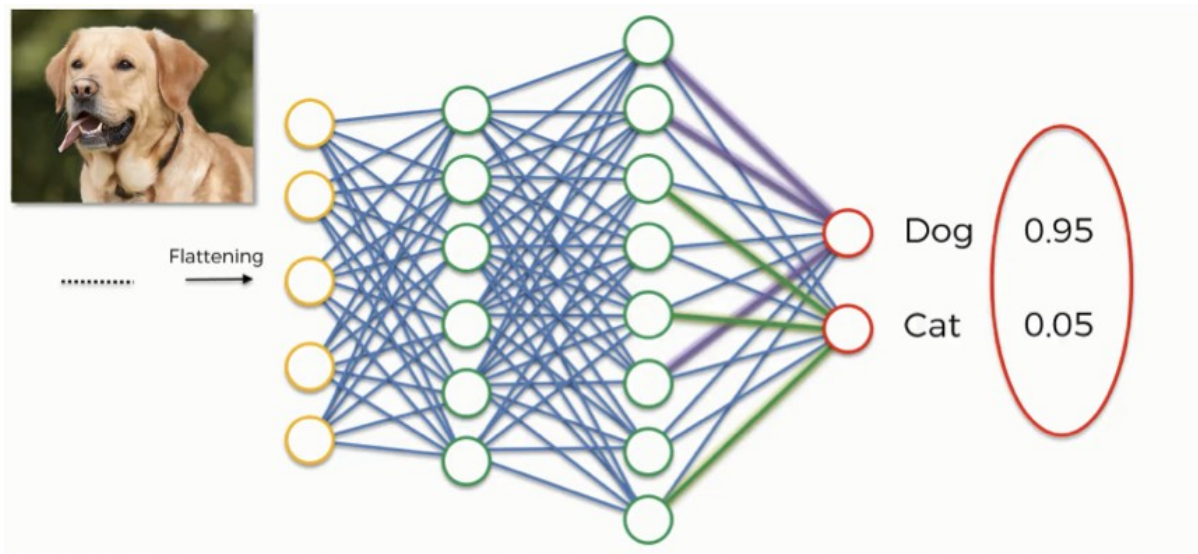
2. Interpretar resultados

- Si estamos utilizando una función de activación **ReLu**, cada neurona tendrá valores en el **rango** $[0, \text{inf}]$ y podemos pensar en seleccionar directamente la neurona final con **mayor activación** como clase.
- Si utilizamos la función **Sigmoidal** partimos de un escenario similar... pero controlado en el **rango** $[0,1]$
- El problema lo tenemos si queremos interpretar el resultados como **probabilidad de pertenencia** de la entrada a cada una de las clases.

3. Softmax

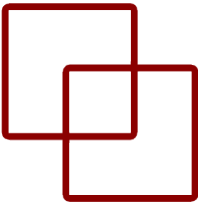


- Para solventar esta carencia se introduce la regresión Softmax (también llamada *Multinomial Logistic* o *Maximum Entropy Classifier*).
- Esta regresión parte de la siguiente premisa:
 - Todas las **clases** o categorías son **excluyentes entre sí**; es ampliamente utilizada en otros campos como SVMs

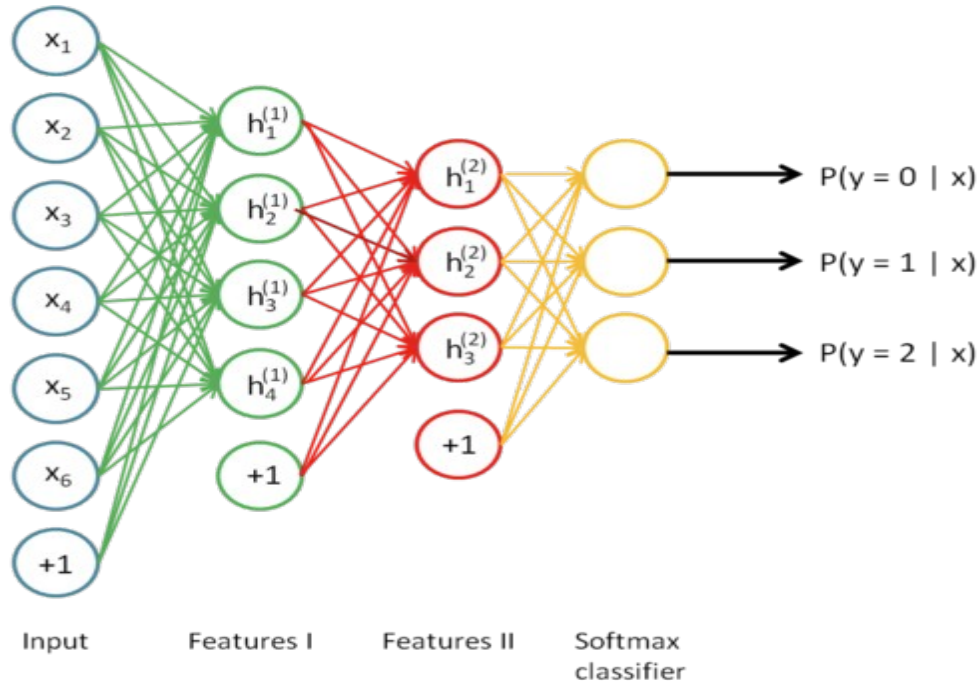


$$\sigma : \mathbb{R}^K \rightarrow [0, 1]^K$$
$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{para } j = 1, \dots, K.$$

4. Clases de salida

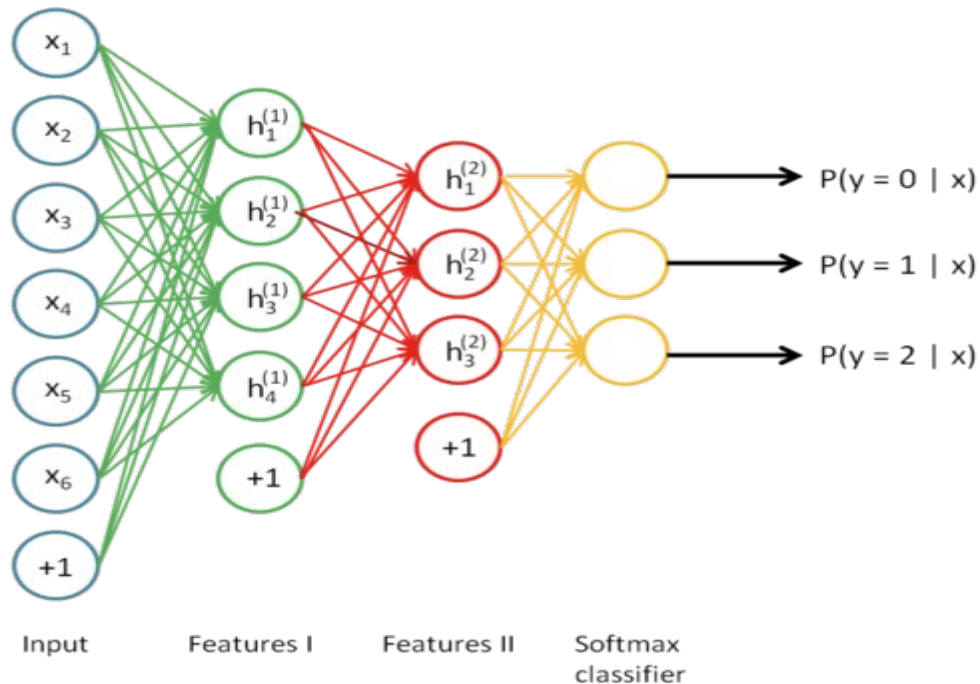


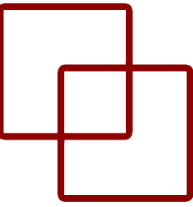
- La fórmula parte de z valores de salida, los cuales se corresponden con las z posibles clases del dominio y generan un nuevo **vector normalizado**.



4. Clases de salida

- La fórmula parte de z valores de salida, los cuales se corresponden con las z posibles clases del dominio y generan un nuevo **vector normalizado**.





5. Interpretación de salida

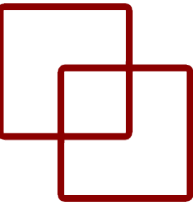
- El nuevo vector obtenido tras el uso de Softmax ya puede **interpretarse** como la probabilidad de pertenencia a cada una de las clases del dominio
 - Para que el uso de Softmax sea efectivo, debemos trabajar con la **función de coste *cross-entropy***, la cual se puede integrar fácilmente en la formulación para realizar el aprendizaje
- En esta función, el término O_i representa la salida i tras el uso de Softmax, y T representa el valor real (1 ó 0)

$$H(T_i, O_i) = - \sum_m T_i \cdot \log(O_i).$$



LAB CTIC  UNI

Gradiente descendiente y Backpropagation

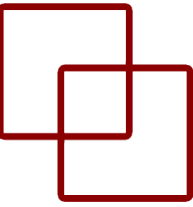


1. Contexto

- Vamos a profundizar en dos técnicas clave:
 - Gradiente Descendiente
 - *Backpropagation*
- Partimos de un **problema de optimización**, donde deseamos reducir un valor en concreto → función de coste.
- La función de coste puede desarrollarse para mostrar la formulación completa, y derivable parcialmente para cada uno de los parámetros a aprender (pesos y sesgo)



Backpropagation

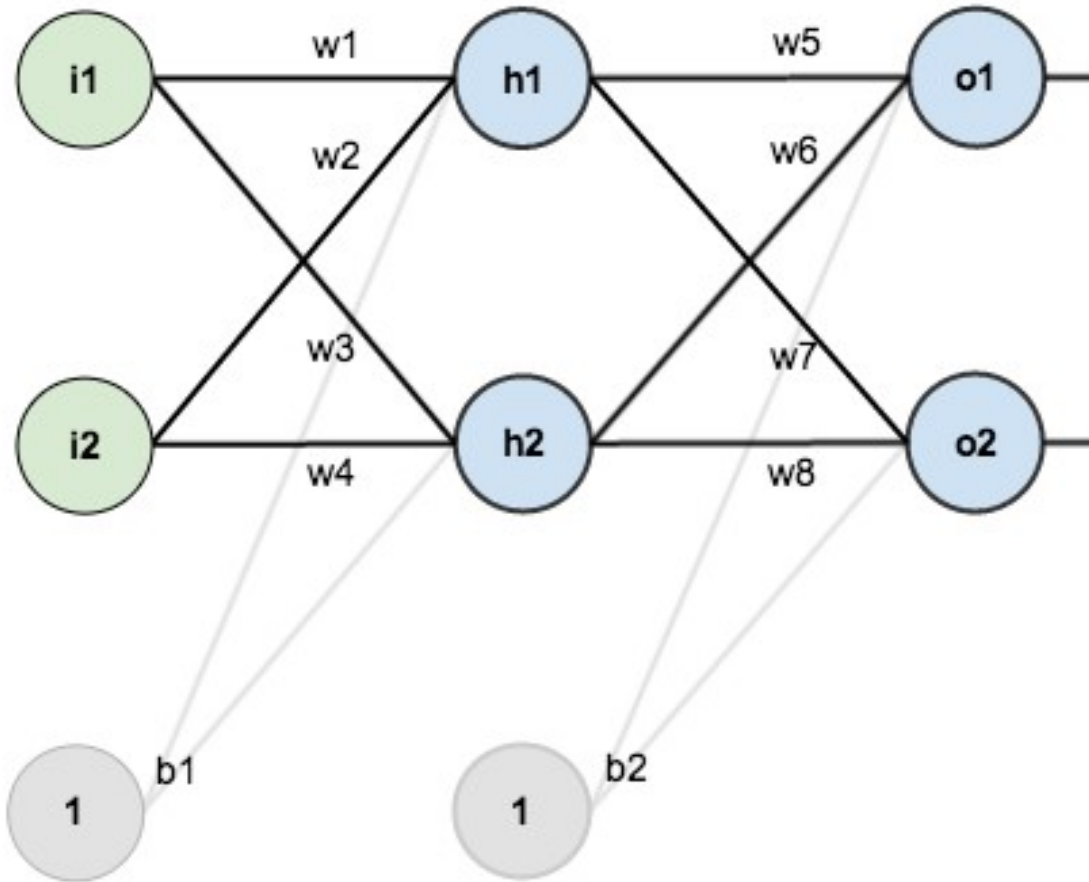
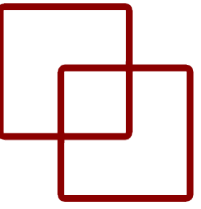


1. Contexto

- En lugar de aprender entrada a entrada, se suele utilizar **pequeños conjuntos de entrada**, lo que se conoce como *mini-batch gradient descent*.
- *Backpropagation* consiste de forma iterativa la corrección de parámetros con Gradiente Descendiente:
 - Para optimizar los cálculos se utiliza la regla de la cadena.
- La aplicación de *Backpropagation* se realizar **capa a capa** desde el final, de forma que los primeros cálculos afectan a la capa de salida y la última de las capas ocultas.
- Para entender sus implicaciones, nos remitiremos a un ejemplo usando la topología de la siguiente página, donde se parte de unos pesos y sesgo ya establecidos.
 - Usamos la función de activación **Sigmoidal** y el **error cuadrático medio** como función de coste
 - <https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/>

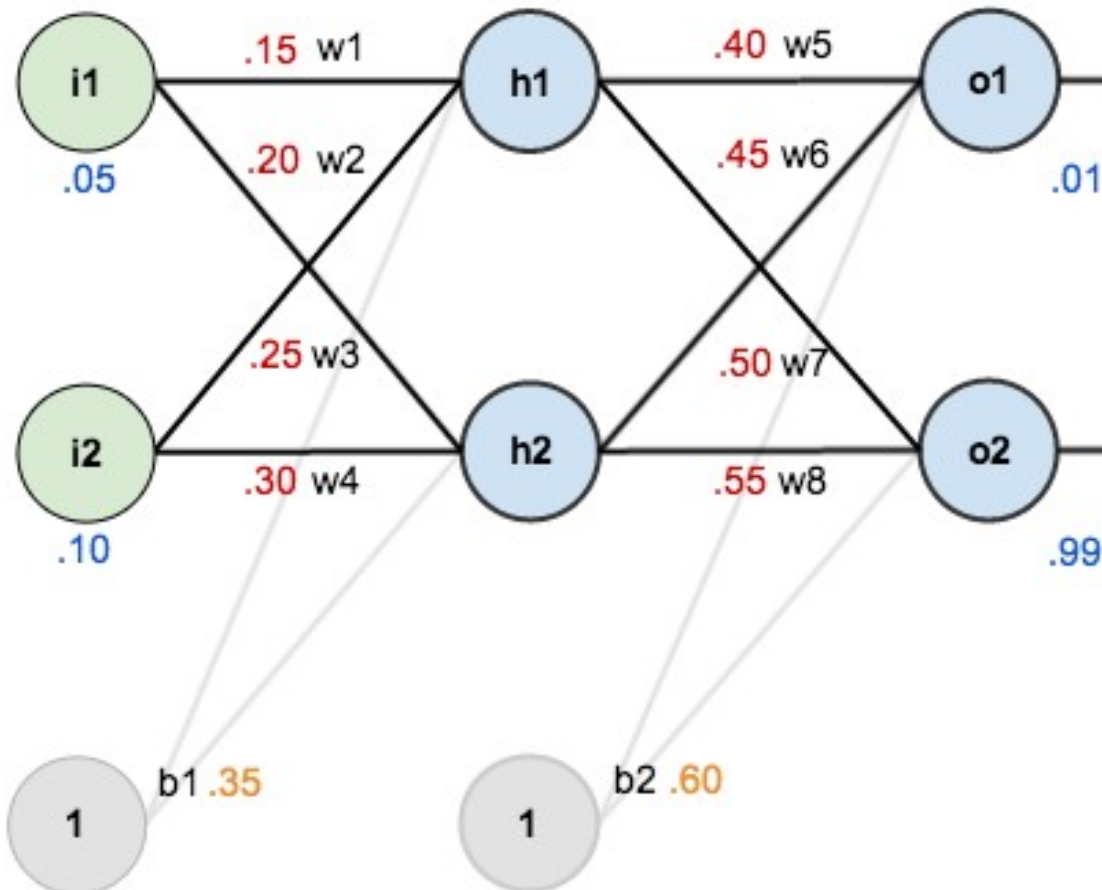
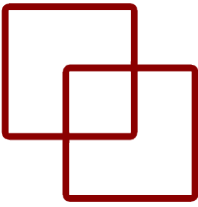
2. Ejemplo

Arquitectura de la red



2. Ejemplo

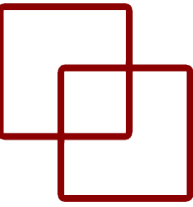
Arquitectura de la red



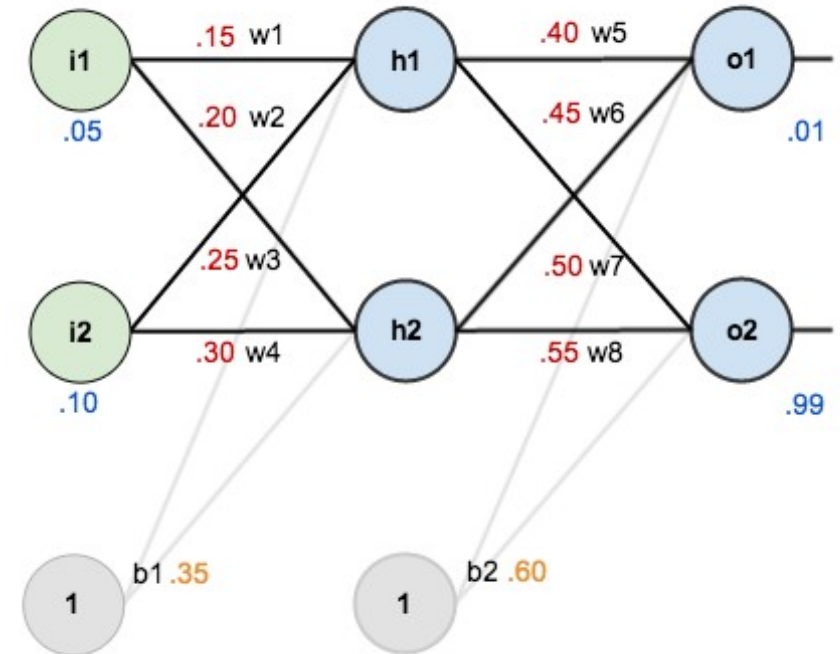
- Dos neuronas de entrada (i)
- Dos neuronas ocultas (h)
- Dos posibles categorías (o)
- w son los pesos
- Abajo están los bías o coste que son únicos

2. Ejemplo

Valores de capas ocultas

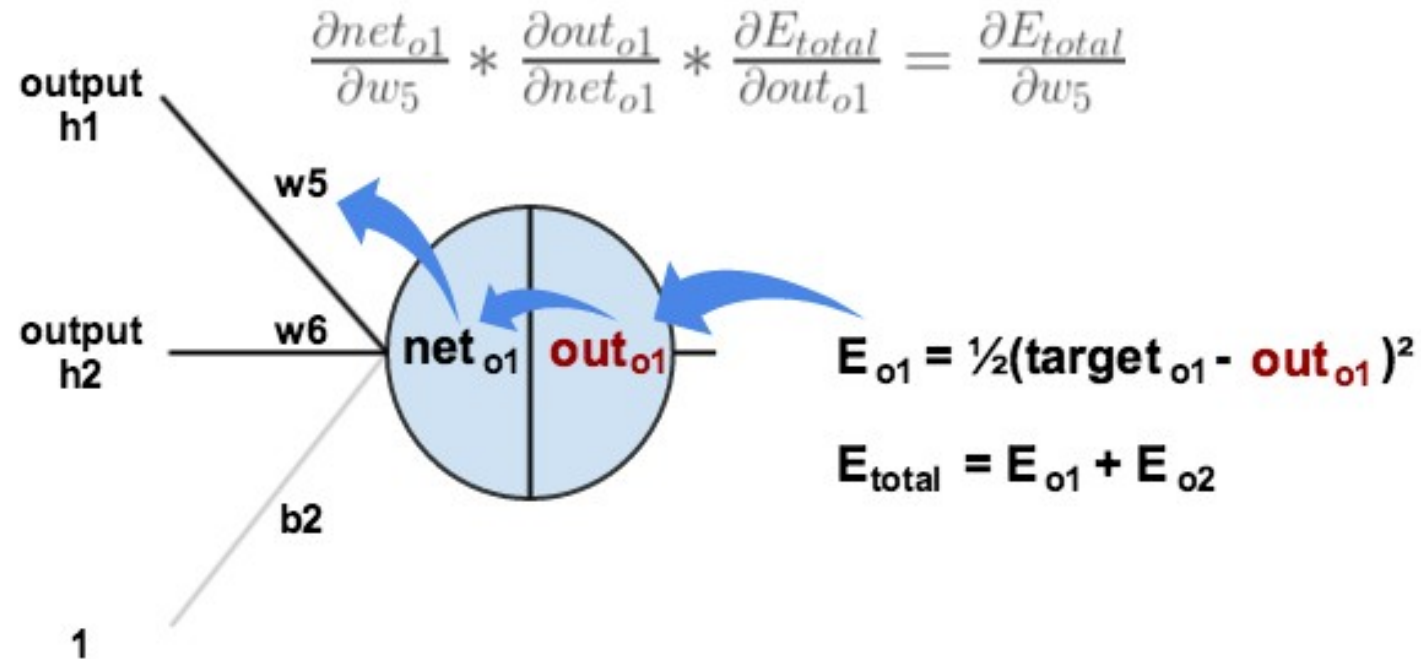
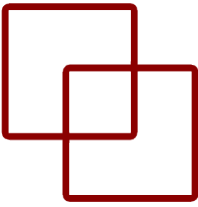


- Centrándonos en la neurona *O1*, estudiaremos el proceso de aprendizaje a partir de la entrada [0.05,0.10], cuya salida esperada es [0.01,0.99]
- El paso **forward**, los valores de *h1* y *h2* se establecen a partir de la suma de pesos por la entrada más sesgo, y en este caso usamos la **función de activación** Sigmoidal:
 - $f(h1) = \text{sigmoid}(0.15 * 0.05 + 0.2 * 0.10 + 0.35)$
 - $f(h1) = \text{sigmoid}(0.3775) = 0.593269992$
 - $f(h2) = \text{sigmoid}(0.25 * 0.05 + 0.3 * 0.10 + 0.35)$
 - $f(h2) = \text{sigmoid}(0.3925) = 0.5958843378$



2. Ejemplo

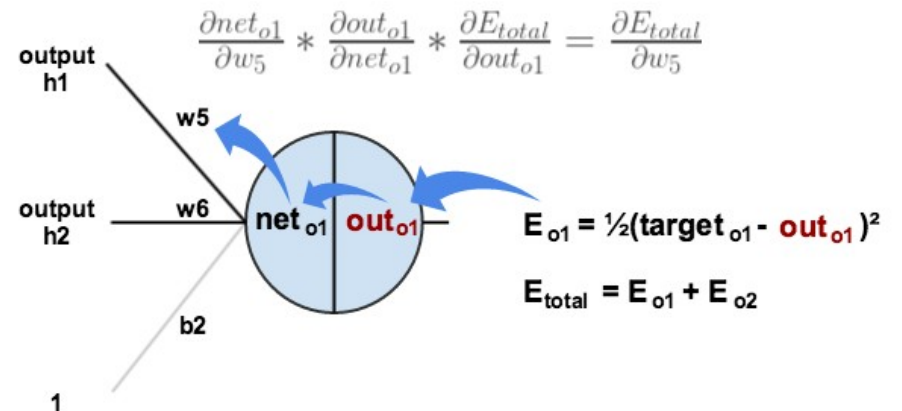
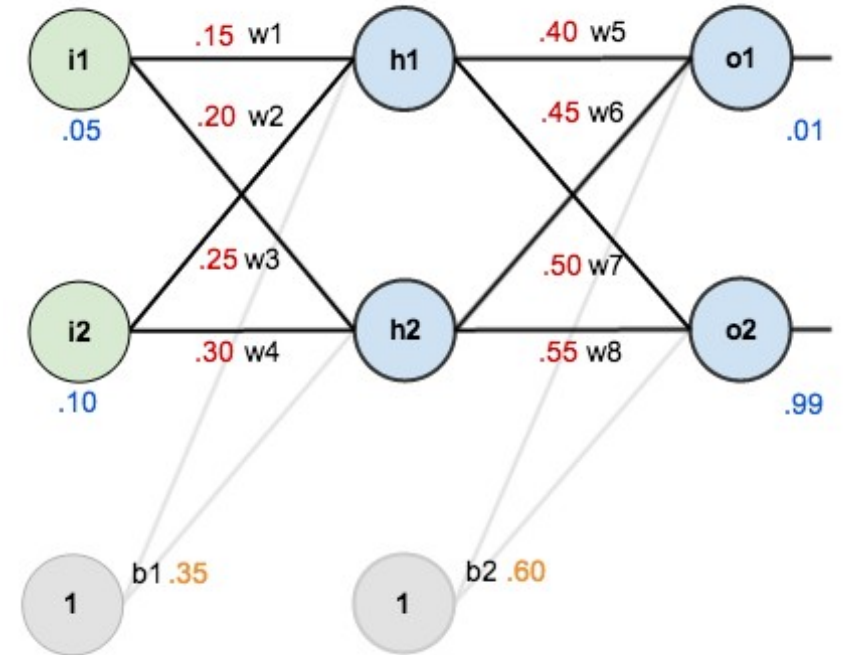
Calcular el error



2. Ejemplo

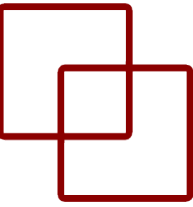
Calcular el error

- Tras ello pasamos a calcular los valores reales obtenidos en la salida de $O1$ y $O2$:
 - $f(O1) = \text{sigmoid}(0.4 * 0.593269992 + 0.45 * 0.5958843378 + 0.6)$
 - $f(O1) = \text{sigmoid}(1.105905967) = 0.75136507$
 - $f(O2) = \text{sigmoid}(0.5 * 0.593269992 + 0.55 * 0.5958843378 + 0.6)$
 - $f(O2) = \text{sigmoid}(1.22437138) = 0.772928465$
- Y calculamos el **error** (utilizando el error cuadrático) ya que no es lo esperado del target:
 - Error total = $E(O1) + E(O2)$
 - $E(O1) = \frac{1}{2} (0.01 - 0.75136507)^2 = 0.274811083$
 - $E(O2) = \frac{1}{2} (0.99 - 0.772928465)^2 = 0.00235560026$
 - Error total = 0.298371109



2. Ejemplo

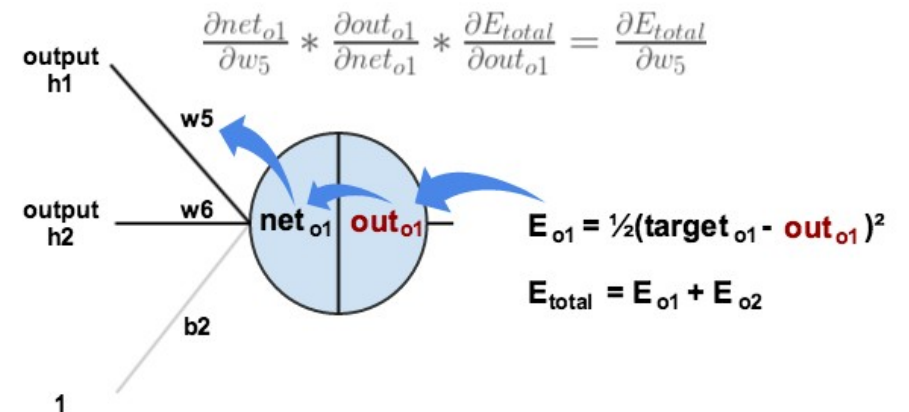
Etapa Backwards



- Como no hemos tenido los valores esperados tenemos que corregir la red con una rectificación para poder aproximarnos al error esperado (target establecido) [0.1,0.99]
- Calculamos el **gradiente** para w_5 (valor inicial 0.4), la cual afecta únicamente al al cálculo de O1
- Hacemos uso de la regla de la cadena para descomponer, y obtenemos lo siguiente:

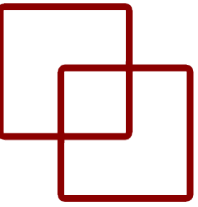
$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

- El objetivo es **simplificar** el cálculo en pequeñas partes que también se pueden **reutilizar** en otros cálculos

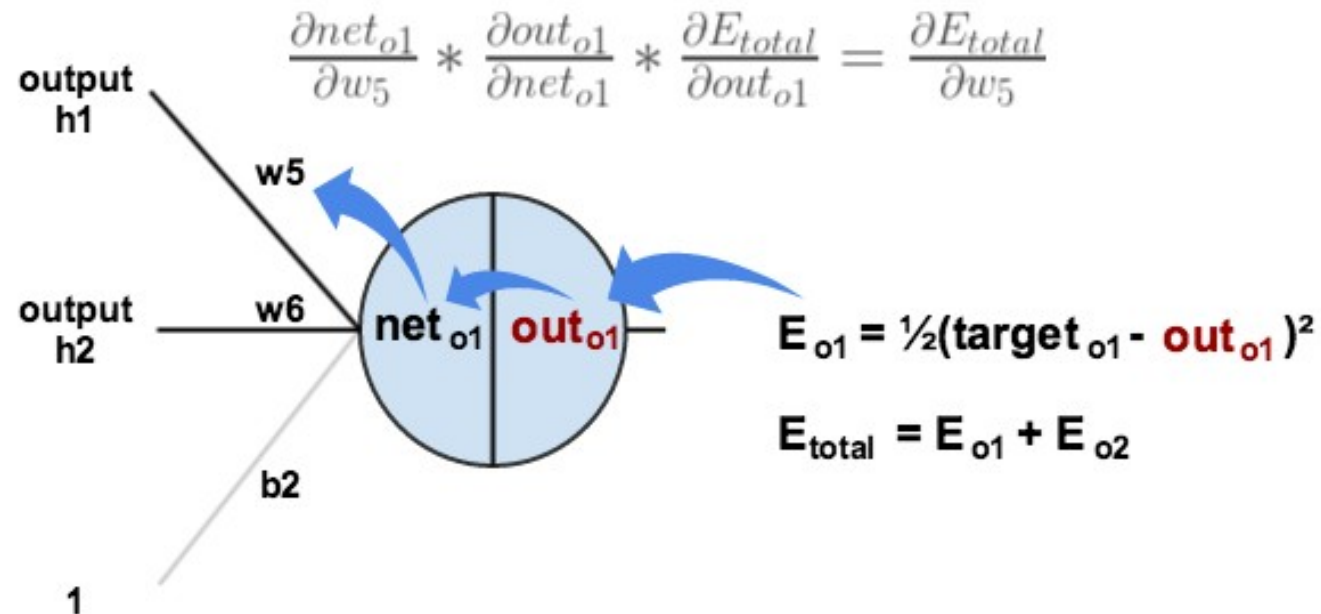


2. Ejemplo

Etapa Backwards

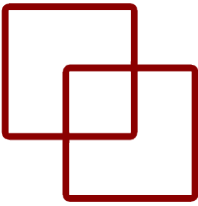


- La regla de la cadena se representa visualmente en la siguiente Figura, donde podemos observar los **tres pasos** que intervienen.

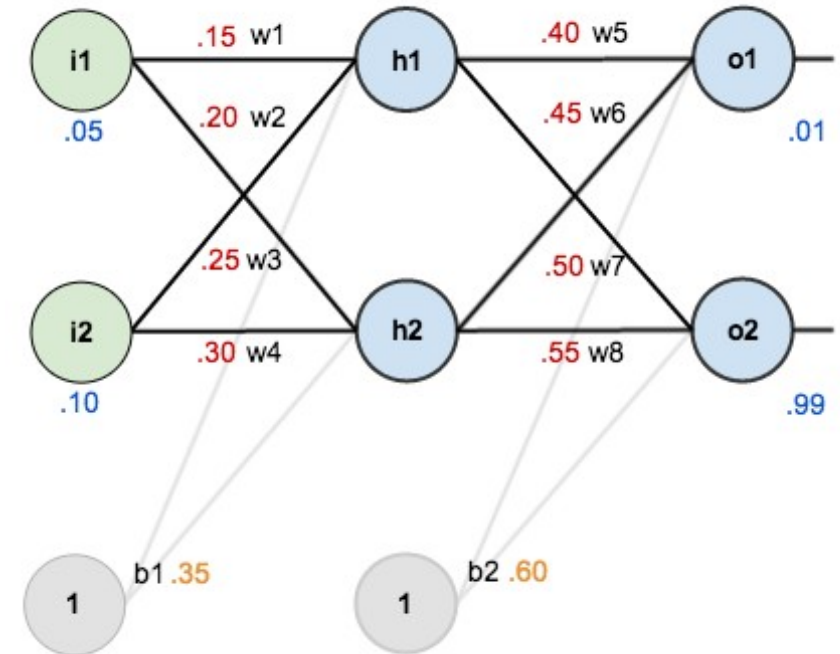


2. Ejemplo

Etapa Backwards



- Tras ello pasamos a calcular los valores reales obtenidos en la salida de $O1$ y $O2$:
 - $f(O1) = \text{sigmoid}(0.4 * 0.593269992 + 0.45 * 0.5958843378 + 0.6)$
 - $f(O1) = \text{sigmoid}(1.105905967) = 0.75136507$
 - $f(O2) = \text{sigmoid}(0.5 * 0.593269992 + 0.55 * 0.5958843378 + 0.6)$
 - $f(O2) = \text{sigmoid}(1.22437138) = 0.772928465$
- Y calculamos el **error** (utilizando el error cuadrático) ya que no es lo esperado del target:
 - Error total = $E(O1) + E(O2)$
 - $E(O1) = \frac{1}{2} (0.01 - 0.75136507)^2 = 0.274811083$
 - $E(O2) = \frac{1}{2} (0.99 - 0.772928465)^2 = 0.00235560026$
 - Error total = 0.298371109



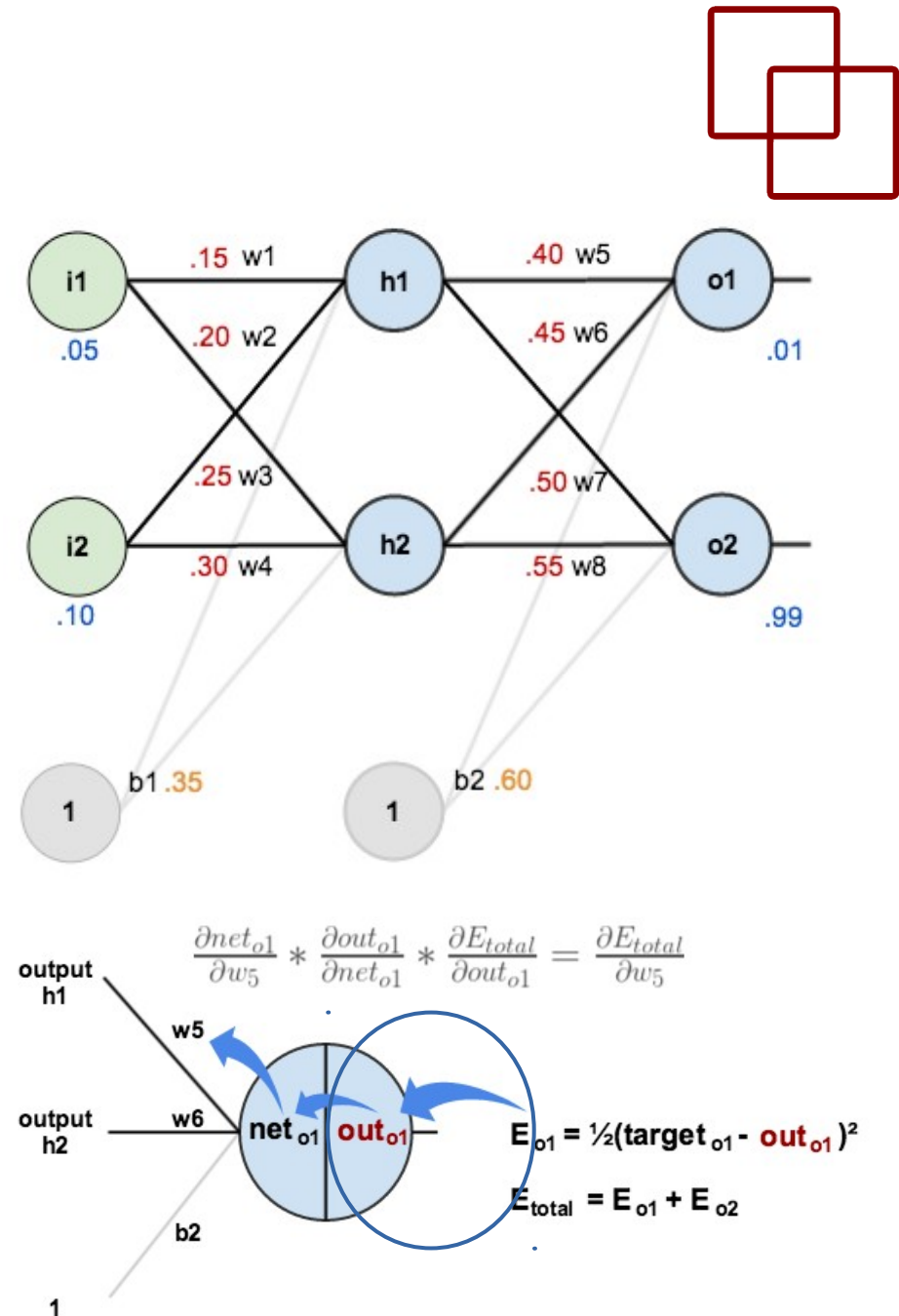
2. Ejemplo

Calcular el error en backward

- Empezando desde el final, computamos la **derivada parcial** del error total con **respecto a O1**.

$$E_{total} = \sum \frac{1}{2}(target - output)^2$$

- Derivando las fórmulas usadas anteriormente:
 - $E(O1) = \frac{1}{2} (t_{O1} - O1)^2 \rightarrow \text{derivar}(O1) = 2 * \frac{1}{2} (t_{O1} - O1)^{2-1} * (-1)$
 - $E(O2) = \frac{1}{2} (t_{O2} - O2)^2 \rightarrow \text{derivar}(O1) = 0$
 - $E(\text{total}/O1) = O1 - t_{O1} = 0.75136507 - 0.01 = 0.74136507$
(donde O1 estaba previamente calculado)



2. Ejemplo

Calcular el error en backward

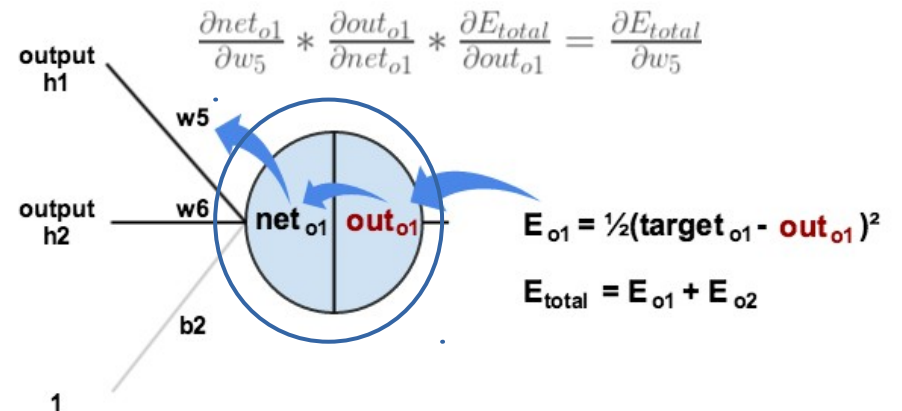
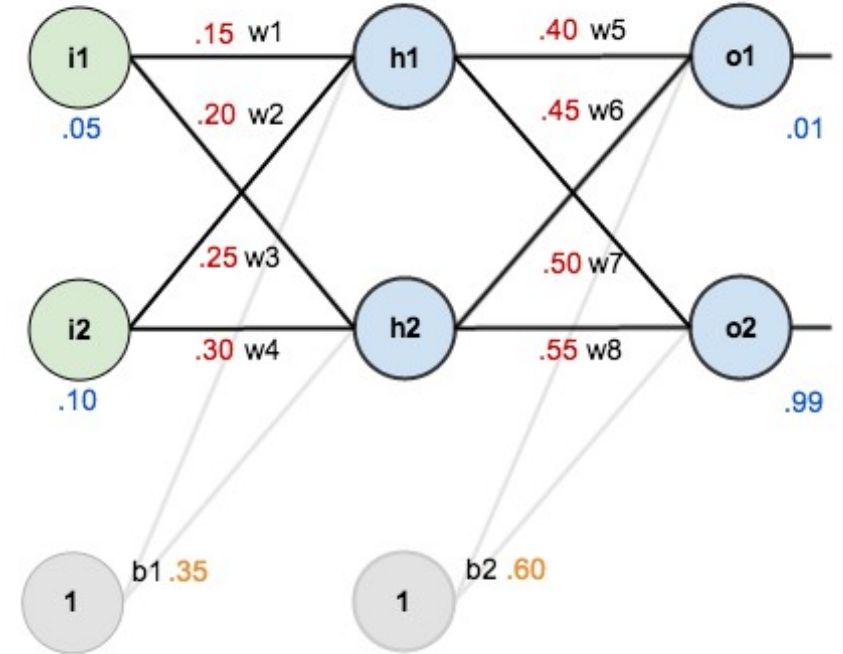
- En la siguiente etapa calculamos el gradiente de la neurona $O1$ con respecto a su entrada (net_{o1}). Al estar usando la Sigmoidal entonces:

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}}$$

- $O1 = 1/(1+e^{-net_{o1}}) \rightarrow \text{derivar}(net_{o1}) = O_1 * (1 - O_1)$
- $(O_1 / net_{o1}) = 0.75136507 * (1-0.75136507)$
- $(O_1 / net_{o1}) = 0.186815602$
 - De nuevo usamos un valor ($O1$) ya computado

* Derivada parcial de esta función: su valor multiplicado por 1 menos su valor

* Funciones ReLu: la derivada parcial es 0 ó 1 para valores negativos o positivos



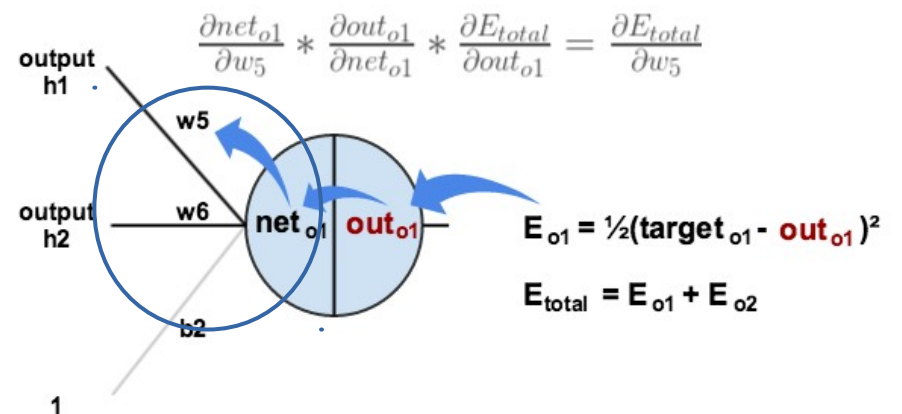
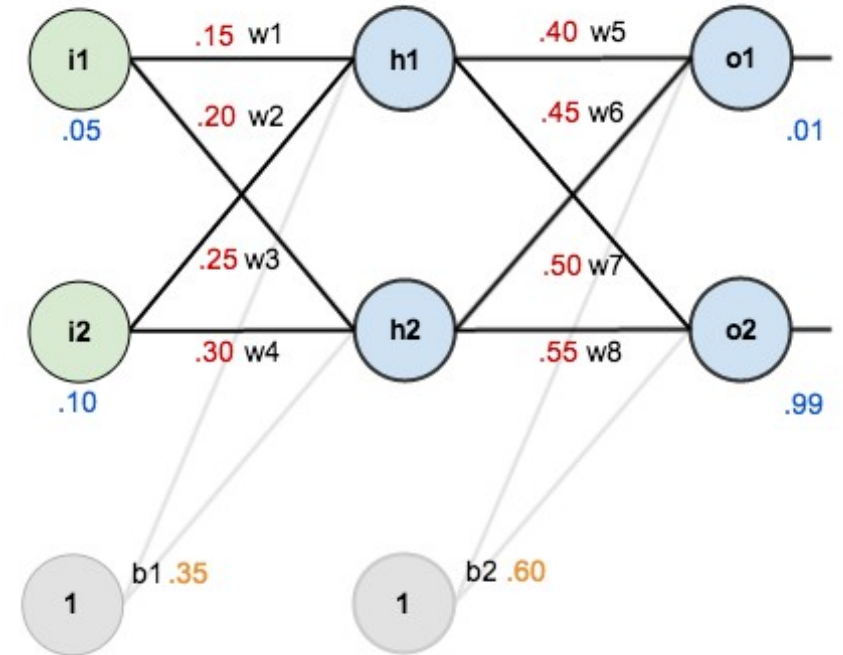
2. Ejemplo

Calcular el error en backward

- Finalizamos con el gradiente de la entrada de net_{o1} con respecto al peso w_5

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

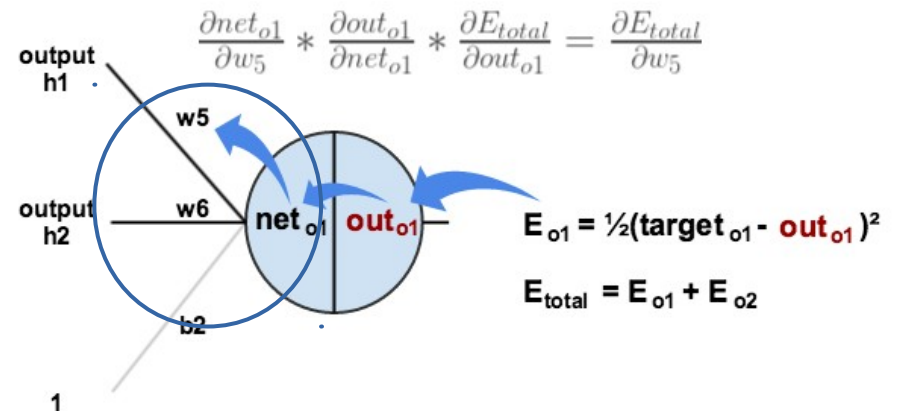
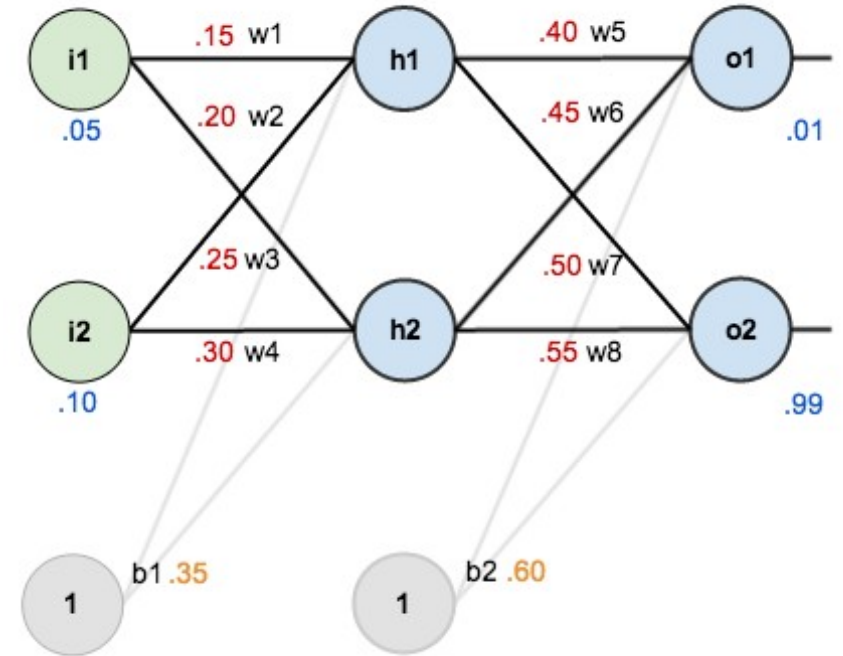
- $net_{o1} \rightarrow \text{derivar}(w_5) = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1}$
- $(net_{o1} / w_5) = out_{h1} = 0.593269992$
(valor ya calculado)



2. Ejemplo

Calcular el error total

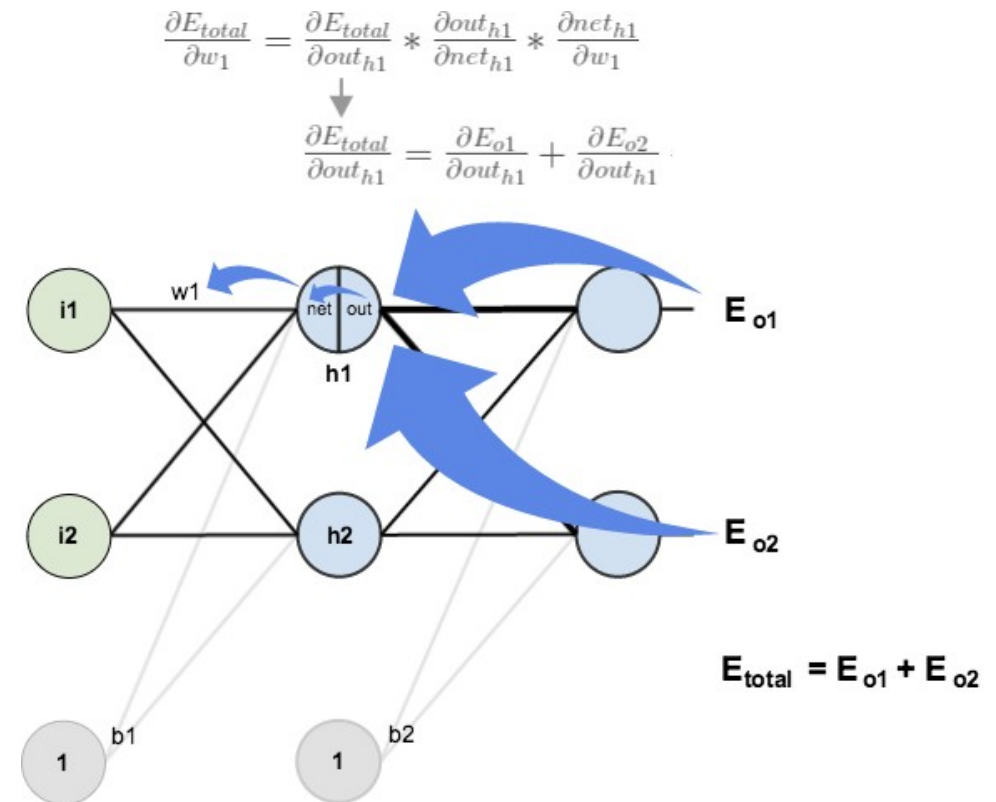
- Juntamos todo y obtenemos:
 - $E(\text{total} / w_5) = E(\text{total} / O_1) * (O_1 / \text{net}_{o1}) * (\text{net}_{o1} / w_5)$
 - $E(\text{total} / w_5) = 0.74136507 * 0.186815602 * 0.593269992 = \mathbf{0.082167041}$
- Este valor si sería el usado para corregir w_5 utilizando un **ratio de aprendizaje**, de forma que tendríamos:
 - $w_5 (\text{corregido}) = w_5 - \text{ratio} * E(\text{total} / w_5)$
 - $w_5 (\text{corregido}) = 0.4 - \text{ratio} * 0.082167041$
 - Usando, p.e. 0.05 de ratio, tendríamos **0.395891648**



2. Ejemplo

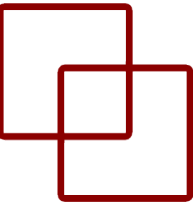
Corrección de parámetros

- Tras la actualización de w_5 , el proceso completaría una **vuelta completa backwards** hasta la corrección de todos los parámetros de la red
- En este paso, la red habrá mejorado algo, pero será necesario **repetir el mismo proceso** sobre el conjunto de entrenamiento numerosas veces si queremos que el resultado sea aceptable.
 - La descomposición gracias al uso de la regla de la cadena facilita mucho los cálculos a realizar, permitiendo reutilizar muchos parámetros de la etapa *forwards*.





Conclusiones



1. Conclusiones

- En esta sesión hemos visto diferentes conceptos que nos permiten tener un mayor conocimiento de las redes neuronales, necesarios para sesiones posteriores.
- Tal y como se puede intuir, cualquier red dispondrá de numerosos parámetros a aprender, por lo que este proceso será complejo y requerirá tiempo.
- Los parámetros a establecer por parte del usuario de la red serán:
 - Su topología: tipo, número de capas intermedias y la conexión entre ellas.
 - La función de activación
 - El ratio de aprendizaje, o
 - El uso de la función Softmax en la capa de salida

¡GRACIAS!



Dr. Manuel Castillo-Cara
Deep Learning con Python y Keras
Web: www.smartcityperu.org